Dive into Scala

March 8, 2026

Contents

I ELEMENTS OF SCALA 6

1 First Steps 7

2 Scala() 12

3 Classes & Objects 18

4 Collections, Blocks & Iterators 25

5 Standard Types 34

6 Method Madness 42

7 Expressions 47

8 Exceptions, Catch & Throw 56

9 Traits 60

10 Basic IO 64

11 Threads & Processes 70

12 Testing 75

II Scala Surroundings 78

13 Scala and Its World 79

14 Interactive Scala **85**

15 Documenting Scala **90**

16 Package Management with Scala **94**

17 Scala & the Web **100**

18 Extending Scala **111**

III Scala Distilled 116

19 From the Ground Up **117**

20 Core Constructs **128**

21 Functional Things **132**

22 Classes & Objects **135**

23 Data Safety **138**

24 Reflection & Metaprogramming **141**

IV References & Appendices 144

Preface

From it's inception, Scala has been a constant, quiet powerhouse. While the Java ecosystem was still finding its legs, Scala was already looking twenty years into the future. It has consistently attracted the brightest minds in computer science—not just for the sake of academic theory, but to solve the most grueling problems in distributed systems and data.

It was not until more recently that I took the time to take it for a spin. All I can say is that I should have taken a closer look years ago. The language is maximally clean and clear, with a focus on versatility and practical problem solving. It takes everything I loved about the mathematics and computer science academics – clarity, precision and intelligence – and married it with a very easy to understand syntax. In terms of paradigms, it took a radically pragmatic approach, refusing to pick a side in the war between Object-Oriented and Functional programming, choosing instead to marry them. in a way that is very intentional, allowing each to naturally develop in the areas it was best suited. All of this translates into a language that allows you to focus on the problem you are trying to solve, fading into the background while acting as guardrails to guide you to solutions that are clear & well structured. It helps you become a better developer: giving you the chance to write cleaner, more expressive code for your users *and* for future maintainers, not for the compiler.

The tragedy is that Scala has always been better at being a language than talking about it. For too long, the barrier to entry was a lack of clear, iconic messaging, not the language itself. It took me an embarrassingly long time to take a look at this language – scala has been around almost as long as the JVM, but I only started taking a look a couple of years ago – but since picking it up I use it for every little programming job I have now. I should have known this would be the case.

Scala is a language of very high repute in the academic world–it is one of the most heavily researched by post-grad Computer Science researchers. It has immense staying power, with a very robust ecosystem surrounding it. There is now a library for most every task you could think of. And it's used in some of the most high demand developer jobs anywhere in high volume, precision-critical tasks worldwide (after all, scala derives it's name from SCAlable LAnguage).

One issue scala currently suffers from is quality documentation and a coherent framework around it's latest features. That's where this book

comes in.

A Language for the Next Era

Scala is often described as "difficult," but that reputation is a dusty artifact of a bygone era. If anything, Scala likely feels familiar today, since the rest of the world has finally caught up. Features that Scala pioneered years ago have now become the "modern" additions to languages like Java, C#, and Python.

Scala isn't just a language; it's a harbinger. It's the "fine wine" of the JVM—sophisticated, nuanced, and better with age.

Why This Book? Why Now?

We are living in the age of the LLM. When AI can handle the boilerplate and the "how," the human developer's most important job becomes the "what." In this environment, the language you choose should be the one that offers the most precision and the least friction for your ideas.

There has never been a better time to be picky about your backend language. We believe that once you see the elegance of Scala's pattern matching and the purity of its class syntax, you won't want to go back.

How to Read This Book

In the spirit of the great pragmatic manuals, we aren't going to bury you in category theory on page one. Scala is a language of action, and this book is designed to be a conversation between your keyboard and the compiler.

The book is organized into three distinct movements:

Part I: The Elements of Scala

We begin by getting you up and running immediately. You won't find dry lists of keywords here. We will give you enough to get going, in terms of terminology & concepts. We will give an overview of key features, with enought detail of the fundamental building blocks—Types, Functions, and Objects—as you will need to get moving. You will learn to write concise

and clean object-oriented code. You will learn to write "Honest" code using immutable data and pattern matching, moving from a single script to a cohesive application that interacts with your OS and environment. We will finish out by outlining how scala testing is done.

Part II: The Scala Surroundings

A language does not exist in a vacuum. In this section, we step into the "Pragmatic Stack." We explore how Scala interacts with the outside world: querying databases with \textbf{Doobie}, driving web services with \textbf{Tapir}, and performing high-performance HTTP calls with \textbf{sttp}. We also bridge the gap to the Java ecosystem, ensuring you can leverage the full power of the JVM. We will touch on how scala packages things so that developers write modules that play nice with each other.

Part III: Scala Distilled

Once you are comfortable with the "How," we move to the "Why." This part is the deep dive into the engine room. We revisit the language constructs—exploring the nuances of the Type System, Metaprogramming, and Safety Modes. This is where we distill the philosophy that makes Scala "The Scalable Language."

The Appendices: The Developer's Atlas

At the back of the book, you will find a reference to the Standard Library, a dictionary for those coming from other languages like Ruby or Java, and a guide to the tools that keep the Renaissance ecosystem humming.

Whether you are a veteran looking for a more expressive tool or a newcomer looking for a solid foundation, this book is designed to meet you where you are. Let's get to work.

Part I

ELEMENTS OF SCALA

Chapter 1

First Steps

Let's get you get scala up and going so that you can follow along with the code as we go.

The Sandbox Setup

Scala is a moving target. Because it is so actively researched, the environment around it evolves quickly. Usually, this leads to a "setup tax"—an hour of fighting environment variables before you write a single line of code.

We're going to skip that.

Instead of installing Scala across your entire Operating System, we're going to build a **Portable Sandbox**. This keeps your project self-contained and your system clean.

You only need two things to get started. Create a folder for this book (let's call it *scala-sandbox*) and follow these steps:

1. **Setup The Engine (The JDK)**: Grab a "Portable" or "Archive" version of the Java Development Kit (we recommend **Eclipse Temurin** from adoptium.net, which can be found at https://adoptium.net/en-GB/temurin/releases. Make sure to grab **.ZIP** file for the JDK for your platform - Windows (or whatever platform you're using)). Make a subdirectory under the root called "jvm", unzip the ZIP file and put it directly into that folder (you should end up with a directory something like "scala-sandbox\jvm\jdk-25.0.1+8", with a bunch of folders in it like "bin", "conf", ...).

2. **The Pilot (Scala CLI)**: Download the **Scala CLI** binary (from VirtusLab's repo, eg. https://github.com/VirtusLab/scala-cli/releases/download/v1.12.2/scal cli-x86_64-pc-win32.zip). This is the Swiss Army Knife of modern Scala. Download the zip file and extract it, then move it into the bin subfolder (eg. scala-sandbox\bin).

The Launch Script

To make this environment "alive," we'll create a small script. This tells your computer to use the local Java and Scala versions you just downloaded, rather than searching the whole house for them. Put the resulting code in a "launch.bat" file in the project root.

For Windows (launch.bat):

```
@echo off
set "JAVA_HOME=%~dp0[your-jdk-folder-name]"
set "PATH=%~dp0bin;%JAVA_HOME%\bin;%PATH%"
echo ----------------------------------------
echo Scala Portable Environment Loaded!
echo ----------------------------------------
:: Automatically create our project settings
(
  echo // project.scala
  echo //> using javaOpt "--sun-misc-unsafe-memory-access=allow"
  echo //> using javaOpt "--add-opens=java.base/java.lang=ALL-UNNAMED"
  echo //> using dep com.lihaoyi::pprint:0.9.6
) > project.scala
cmd /k
```

(Note that you will need to substitute in the relative path where you put the jdk directory (eg. set "JAVA_HOME=%~dp0jdk\jdk-25.0.2+10").

(Also Note: For Mac or Linux users, you can achieve the same by sourcing a simple .sh script to export these paths.)

So we should now have something like:

```
/
├── bin
│   └── scala-cli
├── jdk
│   └── jdk-25.0.2+10
│       ├── bin
│       ├── conf
│       └── ...(other jdk stuff)
├── launch.bat
└── ..
```

Writing & Running Scala

Interactive scala

scala-cli console is the modern choice to executing scala interactively. It's a scala console complete with command-line history and job control (we will cover it later in more depth). After loadiing our environment with launch.bat, 'scala-cli console' will run it from the command line. Once it starts, start typing scala code!

```
...\scala-book>scala-cli console
scala> def hiNumbers(n1: Integer, n2: Integer) =
         n1 + n2
scala> hiNumbers(3, 4)
val res1: Int = 7
```

scala-cli console is super useful, and the more you use it the more handy it becomes

Additionally, you can load your local code to test out the code therein:

```
...\scala-book>scala-cli console
scala> :load hello.sc
Keep coding!
Compiling at the speed of light!
// defined class Mood
def response(mood: Mood): String
val sequence: List[Mood] = List(Happy, Caffeinated)
```

```
scala>
```

Scala Programs

Now, let's see why we're here. Create a file named *hello.sc* in your folder and type in the following:

```
//> using scala "3.3"
// Programmer moods
/**
 * Represents a developer's state of mind.
 * * @param intensity How strongly the mood is felt (1-10)
 * @note Use [[Mood.Caffeinated]] for high-performance tasks.
 */
enum Mood:
  case Happy, Curious, Caffeinated
// Have empathy
def response(mood: Mood): String = mood match
  case Mood.Happy       => "Keep coding!"
  case Mood.Curious     => "Let's explore the source code."
  case Mood.Caffeinated => "Compiling at the speed of light!"
// A typical morning
val sequence = List(Mood.Happy, Mood.Caffeinated)
sequence.map(response).foreach(println)
```

Now in your shell, cd to the scala-sandox root and tell Scala CLI to run everything in the current directory:

```
% scala-cli .
```

You should see Scala spring to life, download the necessary dependencies (like pprint), and output:

```
Keep coding!
Compiling at the speed of light!
```

Congratulations. You have a professional-grade Scala 3 environment running entirely inside a single folder. No installers, no registry keys, just code.

Scala Documentation

Documentation of source files isn't just for folks mucking around in your code – it can easily be summoned in the console to see what author's of any code had to say. For example, our Mood enum above is meticulously documented for maximum comprehension. The format is similar to a lot of other lanugaes : `/** ... */`, with some keywords to help.

The secret trick here is that you can easily summon this documentation *in the console* with

```
scala> :doc Mood
Represents a developer's state of mind.
 * @param intensity How strongly the mood is felt (1-10)
Note
 - Use [[Mood.Caffeinated]] for high-performance tasks.
```

However, if you do prefer the nicely marked up we version, you can build it with

```
./scala-cli doc . -o ./my-docs
```

from within your directory, and it will generate a full, searchable docs website in you my-docs folder (open my-docs/index.html in a browser to start navigating).

Give it a try in your command line – it may be a bit off-putting first, but is quite powerful the more you get used to it!

Chapter 2

Scala()

One of the goals of this book was to bring a de-mystified, solid introduction to the reader of the scala language. As we have mentioned, from a layman, outsider view scala is viewed as academic. Not the "well-organized", "well-thought out" sense, but in the "it's very theoretical and inaccessible to non-enlightened folks" sense. This is tragic, and we think can and should be remedied by a prioritized, pragmatic, "work with the code first" approach.

So, now that we have our environment set up, let's build a few small things and take a "high level" tour of the language. We can dissect more rigorously later.

Scala is Object-Oriented

In Scala, everything is an object. Variables. Values. Everything. Of course, there are a ton of languages that make that claim – let's run through what this means for scala.

We use classes to define the "blueprints" of data (that aren't already defined as a base class – with some exceptions). The classic example is a jukebox, where each *song* could be considered a class instance. You could store the name of the song, and the artist, as attributes. We could also define methods, such as "play", that could be formed by the instance. The word object is used interchangeably with class instances.

In scala, these objects usually created by calling the special *constructor* for the class, which is just the class name followed by parameters in parentheses:

```
class Song(val title: String)          // define the constructor
val favorite = Song("Bad Moon Rising") // call the constructor
println(favorite.title)
```

You can see that we define the Song class as taking a String parameter called *title*. We then call that constructor with a string value for the title (“Bad Moon Rising”).

We define multiple class instances by calling the constructor with different parameters.

```
val favorite = Song(‘‘Fire It Up’’)
val secondFavorite = Song(‘‘Smells like Teen Spirit’’)
```

We add methods to a class with the “def” keyword (that you may recognize from other languages like Ruby).

```
class Book(val title: String, val author: String)
  def getIndexInfo() =
    s"Book title: $title. Written by: $author"

val book = Book("The Stand", "Stephen King")
println(book.getIndexInfo()) // -> Book title: The Stand. Written by: Stephen
```

Note there’s no return call needed – the last value is assumed to be the result.

Note also the return string is enclosed in **s"...".** This tells us the string contains variable values (prefixed with a $), along with other substitutions like newline (\n).

Like we said, everything in scala is an object, so variables and literals alike can have the methods invoked on them:

```
favorite.toUpperCase() // -> FIRE IT UP
‘‘Dude’’.toLowerCase()   // -> dude
```

Here’s where it gets a little tricky : you can also create *single* objects, without requiring a class. After all, sometimes, you don’t need a blueprint; you just need a single, specific thing.

For this, Scala has the **Object** keyword—creating a "Singleton" in one line. No boilerplate, no complex patterns.

```
object Jukebox:  // define the object
  val brand = "Vintage 1970"
  def play(song: String) = println(s"The $brand Jukebox plays: $song")
Jukebox.play("My Song") // call the play method on the object
```

Now that we have some multiline code, we will point out something - notice that, after we declare the object, all of the code that belongs to it on subsequent lines is indented. This is a pattern from other languages like python, which allows for cleaner code due to the absence of braces you typically find in C-inspired languages (*cough* java *cough*) – the body of the method does not have braces, or even and "end". The body is simply denoted by being indented.

If you've used Ruby, you'll recognize **Traits** as the smarter sibling of Mixins. They let you "mix in" behavior to a class without the rigid, messy hierarchy of traditional inheritance.

Scala

```
trait Speakable:
  def sayHi() = println("Hi!")
class Human extends Speakable

val baby = Human()
vaby.sayHi()
```

In practice, what this means is that we can add behaviour to classes without a mess of complex hierarchies. Want a function from a class? Just add it in – no need to think about which relative to add it to. Traits can also define & contain state – but we will talk about that later.

Scala is Functional

While objects hold our data, **Functions** transform it. Where other languages shy away, scala full embraces this duality of object orientedness and functional focus.

Scala encourages "chains" of logic where data flows through a pipeline like water through a filter.

```
val rawUsers = List("boop_ai", "sttp_dev", "scala_fan", "doobie_pro")
```

```
val result = rawUsers
  .map(_.replace("_", " ")) // 1. Transform: Swap underscores for spaces
  .map(_.capitalize) // 2. Transform: Uppercase the names
  .filter(_.length > 8) // 3. Filter: Keep the long ones
  .sorted // 4. Sort: Alphabetical order
```

The Underscore (_) is Scala's shorthand for "whatever item I'm currently looking at." It keeps your code incredibly lean.

Scala is a Duality

Case classes are a special class that comes pre-packaged with everything you need: instant instantiation, pretty-printing.

In particular, case class variables can be easily "unpacked":

```
case class Player(name: String, age: Int)
val mario = Player("Mario", 27)

val User(m_name, m_age) = mario    // -> we get m_name = ‘‘Mario’’, m_age = 27
```

We can see that we cleanly now get values from inside the object assigned to variables.

Some Basic Scala

So what does this all mean? What does a "real" Scala service look like? It mixes these concepts into a clean, readable file. Here is a small database interaction using Doobie and our favorite "Functional-Light" patterns.

```
package io.startup.api
import doobie.*
import doobie.implicits.*
import cats.effect.IO
// 1. A Trait for configuration (the "Mixin")
trait DBConfig:
  val xa = Transactor.fromDriverManager[IO](
    "org.postgresql.Driver", "jdbc:postgresql:db", "user", "pass"
  )
```

```
// 2. A Case Class for our data structure
case class Product(id: Int, name: String)
// 3. An Object to hold our logic
object ProductService extends DBConfig: // Mixes in DBConfig trait from above
  def getProduct(id: Int) =
    sql"select id, name from products where id = $id"
      .query[Product]                   // Map the SQL columns directly to ou
      .unique
      .transact(xa)                     // Chain methods to get everything we
```

In just a few lines, we've defined a data model, set up a database connection, and written a type-safe query that maps SQL directly to a Scala object.

Pattern Matching

At a first glance, matching in scala looks much the same: take a variable, define a match, then add cases and how to handle them.

However, Pattern Matching **is what many consider a defining feature of scala.** And that's because it's a lot more robust that in other languages.

One particular strength of scala is the ability to combine "unpacking" with Pattern Matching:

```
mario match {
  case User(name, age) => s"$name is $age years old."
}
```

We get the ability to make decisions based on attributes *inside* the object. This can be used recursively to pull apart members of objects that are themselves objects.

Matches go much further, however, enabling condition checks in each case, unpacking of list elements, and more.

```
user match {
  case User("Alice", _) => "Found the boss!"
  case User(name, age) if age < 18 => s"$name is a minor."
  case User(name, _) => s"Regular user: $name"
}
```

```
val numbers = List(1, 2, 3, 4)
numbers match {
  case head :: second :: tail => s"Starts with $head and $second, then the re
  case head :: Nil            => s"Only one element: $head"
  case Nil                    => "The list is empty"
}
```

We can see here how much scala depends on classes. Moreso than other languages, they are a central element of the language, using class equivalence as major decision-making mechanisms.

A Mathematical Heritage

There's no getting around it - Scala evolved from a very mathematical worldview. Treating code as a language that encapsulates a comprehensive view of the world can lead to some innovative approaches.

The "No-Null" Policy: Options

In most languages, null/nil is a landmine. You never know when a variable is empty until your program crashes.

Scala solves this with the Option type. An Option is a box: it either contains Some(value) or it is None. This forces any code that deals with the value to explicitly state what happens when it's null.

```
val user: Option[String] = Some("Albertus")
val unknown: Option[String] = None
// You are forced to decide what happens if it's empty
println(user.getOrElse("Guest")) // Prints: Albertus
println(unknown.getOrElse("Guest")) // Prints: Guest
```

Chapter 3

Classes & Objects

In some circles, Scala is praised purely for its Functional Programming (FP) chops. But Scala prides itself on being a "unification" of two patterns. It doesn't treat Object-Orientation (OO) like a legacy burden; it treats it as a first-class way to organize code.

Let's start by looking at how Scala takes the "boilerplate" out of objects. Subsequently, we will more formally define val vs var declarations. Finally, we will jump into object creation patterns and static content, and then take a deeper look at how class hierarchies are handled, along with with privacy level modifiers.

The Lean Class

Let's start by imagining we want to model a portfolio of companies for an imaginary trading firm. We are interested in knowing how the companies we are tracking are performing, so we will create a company class that records the company's name and revenue over the last year. In many languages, creating a simple data class requires a constructor, field definitions, and assignments. In Scala, the class header does all three at once:

```
class Company(val name: String, val revenue: Int)
```

We can use this to create objects for each company we are interested in:

```
val myCompany = Company("saas-o-rama", 7000) // create a company object
pprint.pprintln(myCompany)
```

```
// Result: hello$_$Company@6d8a00e3
```

That output—the "raw dump" that's output to the screen—isn't very helpful. In the Pickaxe tradition, we want our objects to be communicative. Let's override the default toString to give us a professional readout:

```
class Company(val name: String, val revenue: Int):
  override def toString: String =
    s"[Company: $name | MRR: $$$revenue]"
```

We can see we create a toString method to return a string representation for the object. The "override" keyword is there for an unintuitive reason: every class defined inherits a few methods from a fundamental base class. This base class defines several default method that every subclass inherits, one of which is the "toString" method which returns a string. However, because we want to define our own toString, we need to **override** the one we inherited - which we denote by starting with "override" to define the method.

The string we return uses the substitution string format s"", which allows us to use variable values within the string. (eg. $name, $revenue). You will also notice the $revenue is actually prefaced with 2 additional dollar signs (eg. $$$revenue) – this is how we prefix the $revenue value with an actual dollar sign in the final about put (eg. [Company: saas-o-rama | MRR: $7000]).

We covered something subtle here that's worth mentioning as well – we can use the fields passed into the constructor as instance variables, with each getting their own getter and setter (if they are passed in as vars) by default with no additional code.

So we can now write:

```
val myCompany = Company("saas-o-rama", 7000)
pprint.pprintln(myCompany)
// Result: [Company: saas-o-rama | MRR: $7000]
```

To use our new toString methods.

Specialized Roles: Inheritance

Inheritance allows us to create specialized versions of a general concept. A "Startup," for example, is just a Company that (usually) has seed money and a higher burn rate.

```
class StartupCompany(name: String, revenue: Int, val seedMoney: Int)
  extends Company(name, revenue):
  override def toString: String =
    val parentText = super.toString()
    s"[Startup: $name | Seed: $$$seedMoney] (Parent Info: $parentText)"
```

In our StartupCompany, we see that we have specified that we want to get all of the variables and methods of Company by declaring "extends Company(name, revenue)". However, we also would like to know how much the company got in seed funding, so we add that to the fields list.

We would again like to be able to have our own custom string representation, but what we inherited is a good starting point we want to keep. To do this, we can override the toString method again, but this time we will start by getting the inherited method's value with **super.toString()**, and then prefix it with our own goodness (startup name again and seed money). So StartupCompany instances can now be rendered as a string that still makes use of the inherited value:

```
val myStartup = StartupCompany("startup-o-rama", 7000, 80000)
pprint.pprintln(myStartup) // -> [Startup: startup-o-rama | Seed: $80000] (Pa
```

Notice that we pass name and revenue up to the "parent" Company. We don't mark them as val in the StartupCompany header because they are already defined in the base class.

Mutability: The val vs var Choice

In the class signature, you have a choice that defines the "soul" of your object:

- **val (Value):** Creates a read-only field (a "Getter"). This is the Scala default for a reason—immutable data is easier to reason about.
- **var (Variable):** Creates a field that can be changed (a "Getter" and a "Setter").

```
class FlexibleCompany(val name: String, var revenue: Int)
val c = FlexibleCompany("Pivot-Co", 1000)
c.revenue = 5000 // Perfectly legal because it's a 'var'
// c.name = "NewName" // This would trigger a compiler error!
```

Objects & Attributes

In Scala, we take a very open stance when it comes to access level. By default, all fields are *public*. In fact, there is no public keyword.

We can have private members though. We declare like this:

```
class Company:
  private val secret = "hidden"
```

Similarly, protected is also available, which makes visible to the class and subclasses:

```
protected def calculateTax = ...
```

Companion objects (outlined below) can access any internal members of their associated class:

```
class Company private (val name: String)
object Company:
  def create(name: String) =
    new Company(name)
```

A note on virtual attributes:

We can indeed create new getters and setters with custom behavior with:

```
trait Company:
  var revenue: Int
class Startup extends Company:
  private var internalRevenue = 0
  def revenue: Int = internalRevenue
  def revenue_=(value: Int): Unit =
    println(s"Setting revenue to $value")
    internalRevenue = value
```

Where revenue gives use the getter (x.revenue // -> 2000) and the underscore method gives us setter (x.revenue = 4000).

Companion Objects: The Better "Static"

For our company info tracker, it would be nice if we could keep track of the total number of companies (eg. to calculate average values for example).

In Java or C#, you use the static keyword for methods that belong to the class rather than an instance. Scala replaces this with the Companion Object.

If you create an object with the same name as your class in the same file, they become "companions." This is where you put your Factory Methods (methods that build instances for you).

```
class Company(val name: String, val revenue: Int):
  Company.increment()
object Company:  // aka ‘‘Companion’’ object
  private var count = 0

  private def increment(): Unit =
    count += 1
  def totalCreated: Int = count
  // A Factory Method
  def buildBootstrapped(name: String): Company =
    println(s"Building $name with zero outside capital...")
    new Company(name, 0)  // returns a new instance of Company class
// Usage:
val myNewCo = Company.buildBootstrapped("Funco") // returns a new Company ins
```

So here we have done a couple of things we weren’t able to before.

First, we can see we now have a “buildBootstrapped” method. We now use this to get new Company objects, which we can used to populate some values with defaults (such as total starting revenue starting at . When you call Company.buildBootstrapped, you are talking to the companion Object (the singleton). When it returns a value, it gives you an instance of the Class.

Secondly, we can see that the Company class constructor gets some code that is invoked whenever a new instance of the class is made. In this case, it invokes the increment() method on the companion Company global object – which in turn increments the Company global attribute “count”. This allows us to keep track of the total # of objects that have been created.

Thirdly, we can see our count member and increment() member method are private members of the companion object to Company, however the Company *class* has access to them.

Global State and App Configuration

Not every object needs a "class" counterpart. Sometimes you just need a place to store global settings or utility functions that stay consistent across your entire application.

```
object AppConfig:
  val apiBase = "https://api.startup.io"
  var debugMode = true

  def log(message):
    println(s"[INFO] $message")

// Accessible anywhere without using 'new'
if AppConfig.debugMode then
  println(s"Connecting to ${AppConfig.apiBase}")

AppConfig.log(‘‘We finished’’)
```

This is the ultimate Pragmatic tool: it's clean, it's always there when you need it, and it doesn't require complex "Singleton Pattern" boilerplate.

Note the name following the object is just that: the name. Not the a class.

Variables

Scala works much the same as other languages. When an object is assigned to a variable, the variable actually contains a *reference* to the object created. We can assign a class instance to a variable, then if we assign a secondary variable the value of the first, both will point to the same object:

```
val myNewCo = Company.buildBootstrapped("Funco")
val secondCo = myNewCo
myNewCo.name // -> Funco
secondCo.name // -> Funco
```

Scala also gives a built in method to copy

```
val thirdCo = myNewCo.copy(name = ‘‘three co’’)
thirdCo.name // three co
thirdCo.revenue // 0
```

So that's a little more in depth dive into objects. However, objects get even more interesting when you group them into collections.

Chapter 4

Collections, Blocks & Iterators

Scala is the undisputed heavyweight champion of big data. It makes sense, then, that its collections are where the language outshines almost everything else. While other languages treat lists as simple storage, Scala treats them as high-speed engines for transformation.

In our company example, it would be nice if we had a way to track collections of companies so we didn't need to keep a rolodex around. In addition to a list of all companies, it would also be nice if we had a way to keep track of the companies we planned to meet with in the upcoming week. Both of these are containers that hold one or more objects.

Both of these will need similar sets of methods: add a company, remove a company, and give us a list of companies, etc. Let's work on building a ComanyList class to handle these things.

The Big Three: Lists, Vectors, and Maps

In Scala, we typically reach for three primary containers. By default, these are immutable. You don't "change" a list; you create a new, refined version of it.

For our CompanyList, we will explore the first 2 as options.

Lists: The Bread and Butter

A List is an ordered sequence of items, ideal for small to medium-sized data. Each item occupies a spot in the list, referenced by non-negative integers. You can create lists with literals, but you can also create from

existing ones - specifying what should change in the new one.

```
val fruit = List("Apple", "Banana", "Cherry")
val first = fruit(0) // "Apple"
// Prepending (Adding to the front)
val moreFruit = "Dragonfruit" :: fruit
// Updating (Creating a new list with one change)
val kiwiList = fruit.updated(2, "Kiwi")
```

Scala uses () for indexing rather than [], a small departure from C-style languages that hints at Scala's mathematical roots. In general, though, we want to use the lift/get operator which returns a nice option (giving us "Some" / "None" instead).

Scala also provides powerful tools like .patch to perform surgery on your data as you create new lists:

```
// Remove 1 element at index 1, and insert "Lemon" and "Lime"
val citruses = fruit.patch(1, List("Lemon", "Lime"), 1) // Apple, Cherry, Lem
```

Now, like Ruby, we can actually do some overriding of the "()" operator by overriding the "apply" method in subclasses of lists:

```
class SafeList(items: List[String]) {
  // This "overrides" the () syntax
  def apply(index: Int): String = {
    if (index >= 0 && index < items.length) {
      items(index) // Calls the underlying list's apply
    } else {
      "Whoops! Out of bounds."
    }
  }
}
// Usage
val mySafeList = new SafeList(List("Scala", "sttp", "Doobie"))
println(mySafeList(1))  // Output: "sttp"
```

The slice operator is a convenient operator that returns a range of items within a list:

```
// Get elements from index 1 to 2 (exclusive of 3)
val middle = citruses.slice(1, 3)
// Result: List(Cherry, Lemon)
```

As we mentioned, lists are immutable, but we create new lists with different values at specified indices.

There are many other operators for lists that are worth looking into, like take and drop and many others.

Maps: The Dictionary

A Map (often called a Hash or Dictionary) is quite similar to lists as collections of indexed options. Maps, however, store pairs of keys and values. There is no sequencing requirement here; you just need to know the key. The example below creates the map as a literal:

```
val meal = Map(
  "main" -> "Lobster",
  "starter" -> "Escargots"
)
val appy = meal("starter") // "Escargots"
```

Map elements are not ordered in any particular way. However, despite this, they are a very popular data structure in scala.

To create new maps we again make copies from existing ones:

```
val userRoles = Map("Alice" -> "Admin", "Bob" -> "User")
// Replace Bob's role
val updatedRoles = userRoles + ("Bob" -> "SuperUser")
```

We can also add / replace multiple keys at once:

```
val batchUpdate = userRoles ++ Map("Alice" -> "Owner", "Bob" -> "Admin")
```

Again, there are many interesting operations to take list creation further, like "updateWith", and it's worth taking a deeper look when you have time.

Vectors: The Random-Access Specialist

Just a quick note about the last of the big 3 containers: the vectors. If your dataset is enormous and you need to jump to the middle of it instantly, use a Vector. While a List has to walk from the beginning to find the 10,000th element, a Vector can "teleport" there.

```
val trees = Vector("Pine", "Birch", "Maple")
val middle = trees(trees.length / 2)
```

Addtionally, updating an element in a vector is much faster as well (close to constant), so large lists needing replacements are also well accommodated here.

Back to our CompanyList Container

With our newfound knowledge, let's return to our CompanyList container. To make things interesting, let's come up with some operations we would like to implement:

```
append(company) -> list
delete_first() -> company
delete_last() -> company
(index) -> company
with_name(name) -> company
```

Let's consider how to implement this. We need to be able to add to the start of the list, remove from the start and the end, as well as grabbing an element at a given index. These all seem to point to a list. However, on closer thought, deleting from the end of a *list* is actually O(n), since lists are implemented in such a way as to require looking at each element until we come to the end in order to know which is the last.

Vectors, however, have no such restriction, and actually remain close to constant operation time on append, delete from start *and* end, and indexing. so we will go with that.

The one question mark is the capability to return an element by it's name. But as we have pointed out in the intro, scala is very adept at filtering a list by name, so likely a map is not warranted here – we can simply search.

We know we can start by creating a class for the CompanyList. We will give it a list to hold the songs:

```
case class CompanyList(name: String, companies: Vector[Company])
```

We will have the append method add to the list and return a new list instance (maintaining immutability):

```
case class CompanyList(name: String, companies: Vector[Company]) {

  // Add a song
  def add(company: Company): Playlist =
    this.copy(companies = companies :+ company)
}
```

The delete_first and delete_last will similarly return new instances of the Playlist with updated companies vector:

```
case class CompanyList(name: String, companies: Vector[Company]) {

  // Add a company
  def add(company: Company): CompanyList =
    this.copy(companies = companies :+ company)
  // Remove a song by index
  // Returns a pair: (The song pulled out, The updated playlist)
  def popFirst: (Option[Company], CompanyList) =
    (songs.headOption, Playlist(songs.drop(1)))

  def popLast: (Option[Company], CompanyList) =
    (songs.lastOption, Playlist(songs.dropRight(1)))
}
```

Interesting – unlike raw lists, vectors come with a robust set of built in operations, like headOption, drop(), lastOption, dropRight, etc. Like we said, worth looking into!

Our next method will be the index method, which is shortened to the "()" call:

```
case class CompanyList(name: String, companies: Vector[Company]) {
  def apply(index: Int): Option[Company] =
    companies.lift(index)
}
```

Note the we implement the "apply" method here. Scala understands that the shorthand for this is simply "(<index_int>)":

```
val list = CompanyList("Tech", Vector(Company("A")))
println(list(0))     // Works automatically! -> ‘‘A’’
```

At this point, let's introduce some tests. ScalaTest is the very unoriginally-named library we make use of here - with the AnyFlatSpec class (one of the most popular testing mechanisms):

```
import org.scalatest.flatspec.AnyFlatSpec
import org.scalatest.matchers.should.Matchers
class CompanyListSpec extends AnyFlatSpec with Matchers:
  "CompanyList" should "return Some(company) when indexing a valid position"
    val google = Company("Google")
    val list = CompanyList("Tech", Vector(google, Company("Nvidia")))

    list(0) shouldBe Some(google)
  }
  it should "return None when the index is out of bounds" in {
    val list = CompanyList("Empty", Vector.empty)

    list(0) shouldBe None
    list(5) shouldBe None
  }
  it should "correctly pop the first element and return the remaining list" i
    val c1 = Company("A")
    val c2 = Company("B")
    val list = CompanyList("Letters", Vector(c1, c2))

    // Assuming our popFirst implementation from earlier:
    // def popFirst: (Option[Company], CompanyList)
    val (result, updatedList) = list.popFirst
```

```
    result shouldBe Some(c1)
    updatedList.companies should have size 1
    updatedList.companies.head shouldBe c2
  }
```

We use the "should" clause to dictate the class being tested and invidivual test case captions. For actual data checks, we make use of the "shouldBe" assertion to test individual values (paired with options, eg. "Some", "None").

To start with, we do a small amount of setup, specifying that our class extends AnyFlatSpec (allowing us the ability to run it's tests) and using Matchers to inspect the actual values (eg. using shouldBe as mentioned above). We give the apply/index method a couple of tests, as well as popFirst. We should naturally extend this to test all methods, but we will leave that as an exercise for the reader.

One operation is conspicuously missing here though, and so we will examine that next - the "with_name" operation.

The Iterator Block (Higher-Order Functions)

This is where Scala gets exciting. Instead of writing manual for loops with counters and temporary variables, we use Blocks.

Let's continue with our CompanyList, and imagine we want to find a specific company within. We *could write a for loop to iterate through companies, returning when we find a match on name:

```
def with_name(brand: String): Option[Company] =
  val result = for
    c <- companies
    if c.name == brand
  yield c
```

This is kind of a scala-ish approach already, using a comprehension to go through a list. However, there is a better way:

```
class CompanyList(companies: Vector[Company]):
  def with_name(brand: String): Option[Company] =
```

```
companies.find { c =>
  c.name == brand
}
```

The find operator applies this test to each entry in the companies vector, until it finds a match and returns.

Let's break down that { c => ... } syntax:

1. **The Variable (c):** This is a temporary name for "the current element" as Scala walks through the list.
2. **The Arrow (=>):** This separates the variable name from the logic.
3. **The Result:** The find method expects a Boolean. It will return the first element where your block results in true.
   Note on Returns: You won't see a return keyword here. In Scala, the last expression in a block is automatically the return value. It's cleaner, less noisy, and perfectly "Pragmatic."

This approach uses a iterator and block function - 2 super-common concepts in scala.

Flexible Logic: Passing Blocks

We can take this a step further. What if we want to allow the user of our class to decide how to find a company? We can define a method that accepts a code block as a parameter.

```
def findFlex(check: Company => Boolean): Option[Company] =
  companies.find { c => check(c) }
```

The type Company => Boolean tells the compiler: "This method expects a function that takes a Company and returns true or false."

Since it's a parameter with the name "check", you can see we invoke by simply calling the parameter name with parameters (eg. check(c)).

What this now allows is for us to call it with whatever logic we want:

```
// Long form
myList.findFlex { c => c.name == "Reebok" }
// The Professional Shorthand
myList.findFlex { _.name == "Reebok" }
```

The Underscore (_): This is the ultimate Scala shorthand. It means "the current item." If you only use the variable once in your block, you can toss the c => and just use the underscore. It turns your code into a succinct, readable sentence.

Again, many methods for collections in scala. A couple are foreach and map (which we have already mentioned), but take a look at docs for more!

A couple of other useful methods are foldLeft and reduce:

```
val totalValuation = companies.foldLeft(0) { (sum, c) => sum + c.valuation }
```

We can see foldLeft is similar to the "inject" method in ruby, where we accumulate over successive calls as we go through the list. Here we start with the initial value of 0.

Reduce and reduceOption are similarly some methods to be aware of. They work like foldLeft, however they don't take an initial value, but rather start with the value from the first element.

```
// Only works if companies is NOT empty
val total = companies.map(_.valuation).reduce(_ + _)
// Safer version
val totalOpt = companies.map(_.valuation).reduceOption(_ + _) // Returns Opti
```

Chapter 5

Standard Types

Scala is more than just a high-level language; it's a language built for performance. Because it runs on the JVM, it gives you direct control over how your data is stored. You don't have to worry about these details every day, but when you're processing a billion rows of data, you'll be glad Scala doesn't hide the "plumbing".

But we really haven't really touched on the basic building blocks of all this : standard types. We inherit much of this from Java, but a refresher might be helpful.

Numbers: Choosing Your Precision

In many scripting languages, a number is just a "number." In Scala, we choose the right tool for the job. This keeps your code optimized and your memory usage lean.

Scala still makes the determination between different whole number (integer) types, since it determines range of possible values and memory occupied:

Type	Range / Use Case
Byte	-128 to 127 (Ultra-lean)
Int	The standard 32-bit integer. Your default choice.
Long	64-bit integer. Use this for IDs or massive counts.
Double	The standard for decimals and scientific math.
Float	Rather than confining ourselves to fixes # decimal places, floats allow numbe
BigInt	Numbers as large as your RAM will allow (stored as objects of va
BigDecimal	Decimal numbers as large as RAM will allow (stored as objects of v

Characters are a type closely linked to int, and we can quickly convert to int with ".toInt"

```
val letter = 'A'
val code = letter.toInt
```

Note that we can get the digit value value of a character (if it is a digit) with ".asDigit".

All numbers are also objects, however, and we can call methods directly on them (eg. (-10).abs).

Strings: The 200-Method Superpower

In Scala, a String is technically a Java String, but it's been "upgraded." Scala breathes life into strings by adding over 200 methods that make text manipulation feel like magic.

Literals and Layouts

You can define string literals with single quotes (for a single Char), double quotes, or triple quotes for multi-line blocks. To keep your code indented without messing up your string's formatting, we use the "Pipe and Strip" trick:

```
var simpleString = ‘‘my simple string’’
val html = """<html>
             | <body>Hello Scala!</body>
             |</html>""".stripMargin
```

The | symbol marks the "margin," and .stripMargin trims everything to the left of it. It's a small detail that makes your source code much more readable. Characters are escaped in the usual way, with a backslash.

Interpolation: Mixing Logic and Text

Why concatenate strings with + when you can bake variables right into them? There are a couple of options here:

- s interpolation: For standard variable injection.
- f interpolation: For "formatted" injection (like rounding decimals).

```
val name = "Renaissance"
println(s"Welcome to $name version ${1 + 1}")
val pi = 3.14159
println(f"Pi is roughly $pi%.2f") // Result: "Pi is roughly 3.14"
```

Notice that both allow us to embed values with the ${...} sequence. f interpolation gives us the added capability to add modifiers for formatting options for different data types, such as decimal places for floats.

You can even combine multiline strings with interpolation for a suped up version of here documents:

```
val tableName = "orders"
val query = s"""SELECT * |FROM $tableName
               |LIMIT 10""".stripMargin
```

Everyday strings

String is probably the most robust built-in class in scala, so we won't cover every method here. Instead, we will look at some common string idioms:

One task almost every dev must work on is deal with lines of text from files that represent structured data. We can getLines from the file, iterating over each, splitting each by the pipe and trimming it down to size.

```
import scala.io.Source
// 1. Load the file (don't forget to close it later!)
val source = Source.fromFile("records.txt")
```

```
try {
  val records = source.getLines().map { line =>
    // 2. Split by pipe and trim each part
    val parts = line.split('|').map(_.trim)

    // 3. Map to a Case Class for type safety
    if (parts.length >= 3) {
      Some(Record(parts(0), parts(1), parts(2)))
    } else None
  }.toList.flatten
  records.foreach(println)
} finally {
  source.close()
}
case class Record(id: String, name: String, category: String)
```

We can also scan strings for regex patterns, such as strings of digits:

```
import scala.io.Source
// 1. Load the file (don't forget to close it later!)
val source = Source.fromFile("records.txt")
try {
  val records = source.getLines().map { line =>
    // 2. Split by pipe and trim each part
    val parts = line.split('|').map(_.trim)

    // 3. Map to a Case Class for type safety
    if (parts.length >= 3) {
      val digitRegex = """(\d+)""".r
      // This returns an Iterator of all matches
      val allNumbers = digitRegex.findAllIn(line).toList
      Some(Record(parts(0), parts(1), parts(2), allNumbers))
    } else None
  }.toList.flatten
  records.foreach(println)
} finally {
  source.close()
}
case class Record(id: String, name: String, category: String, num: String)
```

In fact, it would be really nice if we could have an index that we could build that would tell us what songs each word belongs to:

```
case class Song(title: String, artist: String, lyrics: String)
case class SongIndex(index: Map[String, List[Song]]) {
  def lookup(word: String): List[Song] =
    index.getOrElse(word.toLowerCase, Nil)
}
object SongIndex {
  private val WordRegex = """\w+""".r
  def fromSongs(songs: List[Song]): SongIndex = {
    // We group everything into a Map[Word, List[Song]]
    val builtMap = songs.foldLeft(Map.empty[String, List[Song]]) { (acc, song
      // Extract unique words from the lyrics
      val words = WordRegex.findAllIn(song.lyrics).map(_.toLowerCase).toSet

      words.foldLeft(acc) { (innerMap, word) =>
        val existingSongs = innerMap.getOrElse(word, Nil)
        innerMap + (word -> (existingSongs :+ song))
      }
    }
    SongIndex(builtMap)
  }
}
```

We can see there's a lot of techniques in use here: for each song, we use the WordRex "\w+" to grab each word from a song's lyrics, then compress to a unique "Set".

From there we go through each word, check the map for that key - creating a list of existingSongs. Finally, we take the original map, add an entry with the given word as key and the new song appended to the existing list as value.

This just gives a sample of how scala operates, but again reiterates the core values. It's kind of verbose and requires a bit more effort to think through what we want to build, but the payoff is quality and maintainability.

And we're just scratching the surface here – strings are a first class citizen in scala.

Ranges: Memory-Efficient Sequences

A Range is a clever abstraction. It represents a list of values (like 1 to 1000) without actually creating 1000 integers in your memory. It comes in quite useful when dealing with: sequences, conditions, and intervals It only generates the numbers as you need them.

```
val alphabet = 'a' to 'z' // Includes 'z'
val countdown = 10 until 0 by -1 // 10, 9, 8... 1 (Excludes 0)
val odds = 1 to 10 by 2 // 1, 3, 5, 7, 9
```

If you ever need to "solidify" a range into a real list, just call .toList.

Ranges: Robust Intervals

Ranges can prove useful when testing for conditions.

```
val logicRange = 1 to 10
println(logicRange.contains(5)) // true
println(logicRange.contains(11)) // false
val oddRange = 1 to 10 by 2 // 1, 3, 5, 7, 9
println(oddRange.contains(2)) // false (it's in the bounds, but not in the "s
```

Regular Expressions: Patterns as Objects

In many languages, Regex is a separate "mini-language" that feels bolted on. In Scala, Regular Expressions are first-class objects created by adding .r to a string.

```
val EmailRegex = """([a-zA-Z0-9.-]+)@([a-zA-Z0-9.-]+\.[a-zA-Z]{2,4})""".r
```

Because they are objects, you can call methods directly on them, such as matching operations:

```
val text = "We have 10 apples and 20 oranges"
val NumberRegex = """(\d+)""".r
val first = NumberRegex.findFirstIn(text) // Some("10")
val all = NumberRegex.findAllIn(text).toList // List("10", "20")
```

```
NumberRegex.findFirstMatchIn(text) match {
  case Some(m) =>
    println(s"Found '${m.matched}' starting at character ${m.start}")
    // Output: Found '150' starting at character 25
  case None =>
    println("No match found")
}
```

The match object has a number of attributes & methods of use, like the matched string & position.

If we use triple-quoted strings, special characters don't have to be escaped to be used.

The Power Play: Regex in Pattern Matching

This is where the "Pickaxe" style of programming really shines. You can use your Regex directly inside a match statement to extract data instantly.

```
val input = "Contact us at support@startup.io"
input match
  case EmailRegex(user, domain) => println(s"User: $user on $domain")
  case _ => println("No email found.")
```

Match instances also give us before and after methods that return the entire portion of the original string before and after the match itself. As well, we get the following metadata on each match:

```
.start: The index of the first character of the match.
.end: The index of the character immediately following the match.
.groupCount: How many capturing groups were in your regex.
.group(n): Retrieves a specific group by index.
.source: Returns the entire original string being searched.
.before
.after
```

Again, this is just scratching the surface. We have not talked about .replaceAllIn, unapplySeq, split, and the multitude of find methods available as well.

The "Any" Catch-all

Sometimes you don't know what kind of data you're going to get. The Any type is the root of all types in Scala. You can use it to create functions that handle anything, using pattern matching to sort out the types on the fly.

```
def process(input: Any): Unit = input match
  case s: String => println(s"Processing text: $s")
  case i: Int => println(s"Processing number: $i")
  case _ => println("Unknown entity detected.")
```

Chapter 6

Method Madness

In Scala, a method isn't just a sub-routine; it's an interface. How you design your methods determines how much "friction" other developers (including your future self) feel when using your code.

Let's dig into the deteails and have a look at how to make your methods flexible, succinct, and powerful.

Defining a Method

Methods of classes are defined with "def". Java naming conventions are adopted, so class names are capital camel case, methods - lower camel case, etc. One of the unique patterns of scala is we strive for Uniform Access - that is, methods should look like attribute access. So get methods are strictly descriptors of the data; set methods are named with a trailing underscore.

For example:

```
class Item(private var t: String) {
  // Getter
  def thing(): String = t
  // Setter
  def thing_=(newThing: String): Unit = {
    t = newThing
  }
}
```

```
// Usage:
val myItem = new Item("initial")
myItem.thing = "new_thing" // Scala transforms this into:
                           // myItem.thing_=("new_thing")
println(myItem.thing)      // Output: new_thing
```

A couple of things to note here: we gave the Item a private var. This means it creates an internal object instance variable with the given value. But because we specified "var" and not "val", it can be updated (mutable).

The main point here, though, is the thing() and thing_=() methods. "thing()" gives us a getter. "thing_=()" gives us a setter.

Notice that we also declare parameters for the methods. "parameter: type". As well, notice that the setter returns the type "Unit". This serves the same function as returning "void" in Java or C#. Basically, it tells us there is no return value.

Method bodies contain arbitrary scala code. If you really wanted to, you could define classes inside the method that only exist in that context. Static class and companion objects cannot be used here, though, so just be warned.

Variable Length Arguments (Varargs)

Sometimes, you don't know if a user wants to pass one item, ten items, or none at all. Instead of forcing the user to wrap their data in a List every time, you can use the * syntax.

```
def printAll(names: String*): Unit =
  names.foreach(name => println(s"Hello, $name"))
// Usage:
printAll("Alice")
printAll("Alice", "Bob", "Charlie")
printAll() // Valid: results in an empty sequence
```

Within the method, names is treated as a Seq[String]. It's clean, it's readable, and it's very "Pragmatic."

Methods and Functions

scala's "functional" emphasis means that one thing we can is pass functions as parameters to methods, which can then be invoked within the method itself. The syntax for this is "parameter_name: (FunctionParamType1, FunctionParamType2) => ReturnType". It can then be invoked within the function using traditional syntax:

```
// A method that takes a String and a function as parameters
def greet(name: String, formatter: String => String): Unit = {
  // To invoke it, just call it like a normal method/function
  val result = formatter(name)
  println(result)
}
// 1. Passing an anonymous function (Lambda)
greet("Alice", n => n.toUpperCase)
// 2. Passing a pre-defined function
def bracketFormat(s: String): String = s"[$s]"
greet("Bob", bracketFormat)
```

The "Splat" Operator: Exploding Collections

There will be times when you already have a list of data, but you want to pass it into a method that expects Varargs. If you try to pass the list directly, the compiler will complain.

To solve this, we use the Splat operator (: _*). It tells Scala: "Take this collection and explode it into individual arguments."

```
val developers = List("Scala", "Python", "Rust")
// printAll(developers) // ERROR: Expected String, found List
printAll(developers: _*) // SUCCESS: Splats the list!
```

Creating Clean APIs with Tuples

One of the most useful idioms in Scala is combining Varargs with Tuples. A Tuple is a small, fixed-size collection of related items—like a key and a value.

We can use this to create methods that look and feel like a configuration block:

```
def bulkUpdate(pairs: (String, String)*): Unit =
  // Convert those pairs into a Map for easy lookup
  val dataMap = pairs.toMap
  println(s"Updating ${dataMap.size} fields...")
// Usage:
bulkUpdate(
  "email" -> "alice@startup.io",
  "status" -> "active",
  "plan" -> "pro"
)
```

In this example, "email" -> "alice@startup.io" creates a Tuple of (String, String). By accepting a variable number of these, your method starts to look like a native part of the Scala language.

Calling a Method

Calling a method is done by giving the receiver, the name of the method, and optionally some parameters (which can be functions, as the language vigourously encourages).

```
class Greeter {
  // A method that takes a String and a function as parameters
  def greet(name: String, formatter: String => String): Unit = {
    // To invoke it, just call it like a normal method/function
    val result = formatter(name)
    println(result)
  }
}
val g = Greeter()
// 1. Passing an anonymous function (Lambda)
g.greet("Alice", n => n.toUpperCase)
// 2. Passing a pre-defined function
def bracketFormat(s: String): String = s"[$s]"
g.greet("Bob", bracketFormat)
```

Here, we see g is the object receiver, method name is greet, and we pass a string and a function in parentheses.

Method Return Values

As we noted above, every method has a return value. For methods that act like procedures (eg. don't have a usable return value), we designate the "Unit" return type. We can use this to exit a method or function early. However, in scala this is considered bad form. We should strive as much as possible to design our logic to not need these.

If a return statement is not explicitly given, the last value in the method is used as return value.

Default and Named Arguments

To keep your method signatures from growing into unreadable monsters, Scala allows you to provide default values. This lets users omit arguments they don't care about.

```
def configureApp(port: Int = 8080, debug: Boolean = false): Unit =
  println(s"Starting on $port (Debug: $debug)")
// Use the defaults
configureApp()
// Specify only what you need using Named Arguments
configureApp(debug = true)
```

By combining Default Values with Varargs, you can build methods that are simple for beginners but incredibly flexible for power users.

Chapter 7

Expressions

In many languages, control flow is a series of gates and loops—if this, then that; while this, do that. Scala offers those, but it prefers something more elegant. It treats your code like a **transformation** rather than a set of instructions.

A foundation of this approach is that pretty much everything returns a value or object of some sort; pretty much any statement is an expression. This is what allows us to chain methods together. More than that, however, this allows things like conditional statements to be used in assignment.

```
val age = 20
val status = if (age >= 18) "Adult" else "Minor"
println(status) // Output: Adult
```

Custom Operators: Making Code Readable

Scala allows you to redefine operators like +, -, or * by overriding methods on a class. While this sounds like a "scripting" trick, it's actually a way to make your domain logic read like a native part of the language.

```
case class Position(x: Int, y: Int):
  def +(other: Position): Position =
    Position(this.x + other.x, this.y + other.y)
val p1 = Position(10, 20)
val p2 = Position(5, 5)
val p3 = p1 + p2 // Result: Position(15, 25)
```

The snippet above defines the + operator on the class, allowing us to represent a class operation without the baggage of a full word-style method name.

Commands

The string class has many built in surprises just waiting to be taken advantage of. One of the more interesting ones is the ".!" operator, which treats a string as a command-line operation and invokes it as if from a dos shell.

```
import scala.sys.process._
// Just run it and print to console
val exitCode = "ls -al".!
println(s"Command finished with: $exitCode")
```

Assignment

We touched on this before, but it bears repeating: scala offers a wonderful shorthand to adding parameters that the constructor takes and automatically giving them getters and setters. Simply attach the "val" keyword to add a getter, or the "var" keyword to add a getter and setter.

But more to the point, scala provides an override both for "class(x,y) = value" and "val v = class(x,y)". The following example shows how this works:

```
class MyGrid {
  private val data = scala.collection.mutable.Map[(Int, Int), String]()
  // This allows: grid(x, y) = "Value"
  def update(x: Int, y: Int, value: String): Unit = {
    data((x, y)) = value
    println(s"Set ($x, $y) to $value")
  }
  // This allows: val v = grid(x, y)
  def apply(x: Int, y: Int): String = {
    data.getOrElse((x, y), "Empty")
  }
}
// Usage:
```

```
val grid = new MyGrid()
grid(1, 1) = "Start"  // Calls grid.update(1, 1, "Start")
println(grid(1, 1))   // Calls grid.apply(1, 1)
```

The point here is 2-fold: "apply" gives us the ability to basically call the object as a method and update internal state; while the "update" method gives a default operation gives us a default operation to perform when the object is invoked as if it were a function. Wild stuff.

A note on using assignment in a method: we can invoke other methods on the object we are a part of simply by using the method name, though we can achieve the same thing with the "this." operator.

As well, if we want to extract an array into a subset of variables, we can use the array extractor ("_*" syntax):

```
// x gets the first element
// rest gets the remainder as a WrappedArray/Seq
val Array(x, rest @ _*) = fruit
println(x)    // Apple
println(rest) // ArraySeq(Banana, Cherry, Date)
```

Or alternatively, we can do the reverse with the +: operator:

```
val numbers = Array(1, 2, 3, 4, 5)
val first +: others = numbers
println(first)  // 1
println(others.toList) // List(2, 3, 4, 5)
```

Finally, scala has the traditional "+=" operator, but in addition to numerics it can be invoked on mutable collections as well:

```
var count = 10
count += 5 // Same as count = count + 5
println(count) // 15

import scala.collection.mutable.ArrayBuffer
val list = ArrayBuffer(1, 2, 3)
list += 4 // Adds 4 to the existing buffer
println(list) // ArrayBuffer(1, 2, 3, 4)
```

Finally, note that with immutable collections, this still works, but creates an entirely new list holding the collection with the item appended:

```
var items = List(1, 2, 3) // Immutable List
items += 4
```

Conditionals

Scala's conditional checks take the familiar mechanisms as foundational, and lift them to new hights. First, however, let's take a quick look at boolean expressions.

Every boolean expression can be either true or false. Full stop. Not a lot of flexibility. However, with pattern matching, this is rarely what we test for.

Another concept can be viewed as foundational here - namely the dichotomy introduced by the "Option" type. Here, the foundational values are either "Some" or "None".

```
val maybeName: Option[String] = Some("Alice")
maybeName match {
  case Some(name) => println(s"Found: $name") // Extract 'name' here
  case None       => println("No name found")
}
```

We can see that before we examine anything else, we must first check if there is "Some" or "None". This effectively packages "null" checks and other empty states into a coherent idiom that is much more readable. This brings us back once again to the fact that matching leads to much safer code.

Standard boolean operators are the same as traditional ones for the most part. && and || are used to join expressions, and short circuit in the normal circumstances. ! is the traditional "not" test for boolean values as well. (Incidentally, the "!" operator can be overridden with .unary_! method overrides.

Ther are 2 equality operators we can use: "==" tests for equality and returns false if types are not the same, while ".eq(obj)" checks if 2 variables point to the same object exactly. There are some libraries that also define the "===" operator, allowing for typesafe equality checks.

The Option pattern we mentioned above is generally how the language avoids null checks. We use a match on an expression, and act accordingly.

Conditionals and the "Safe" Choice

Since Scala shuns null, we don't use the standard "if-not-null" checks. Instead, we use "coalescing" to provide defaults.

```
val storedName: Option[String] = None
val name = storedName.getOrElse("Guest") // The "Elvis" move
```

In fact, scala has several methods of this form, that check if a value exists, set it to a value if need be, then add to that list:

```
import scala.collection.mutable
val registry = mutable.Map[String, mutable.ListBuffer[String]]()
def addValue(key: String, value: String): Unit = {
  // 1. Get the list or create a new ListBuffer if key doesn't exist
  // 2. Append the value using +=
  registry.getOrElseUpdate(key, mutable.ListBuffer()) += value
}
addValue("user1", "login")
addValue("user1", "logout")
println(registry) // Map(user1 -> ListBuffer(login, logout))
```

A few other notes: if statements for the most part work like any other language. The ternary operator (thing = (a == b) ? 1 : 0) is also supported.

If-Guards

An **If-Guard** is a filter applied directly to a control structure. Instead of wrapping your logic in a giant if block inside a loop, you simply "guard" the entrance:

```
val numbers = List(1, 2, 3, 10, 20)
for
  n <- numbers
  if n > 5 // This is the guard
do
  println(s"Large number: $n")
```

Pattern Matching: The Table of Contents

Pattern matching is the defining feature of Scala. It's not just a "switch" statement; it's a way to deconstruct your data and act on its shape.

```
val toTest: Any = "Success"
toTest match
  case 0 => "It's a zero" // Constant match
  case "Success" => "It's a win!" // Value match
  case s: String => s"It's a string of length ${s.length}" // Type match
  case _ => "Fallback" // The Wildcard
```

Additionally, we can get a "parallel assigment" type functionality using pattern matching on tuples or collections:

```
// Assigning two variables at once
val (lat, long) = (43.45, -80.49)
println(lat)  // 43.45
println(long) // -80.49
```

Destructuring: Looking Inside

The real magic happens when you use matching to "reach inside" an object. This is called **destructuring**.

```
case class User(name: String, role: String)
val user = User("Alice", "Admin")
user match
  case User("Alice", _) => "Hi Alice!" // Ignore the role
  case User(name, "Admin") => s"Alert: Admin $name is here."
  case User(name, role) => s"User $name is a $role"
```

You can even match on the **shape of a List**:

```
val list = List(1, 2, 3)
list match
  case Nil => "Empty"
  case List(x) => s"Just one item: $x"
  case head :: tail => s"Starts with $head, followed by $tail"
```

Case statements are probably the most important concept in scala. Meditate on this and embrace it in your heart.

The Loop Evolution

In procedural programming, we use loops to do things. In Scala, we use them to evolve data.

The Side-Effect Loop

We use **do** when we want to interact with the outside world (I/O, printing, databases). This is where the loop has a "side effect."

```
val apps = List("sttp", "Doobie", "Tapir")
for app <- apps do
  println(s"Learning $app...") // printing to stdout
```

The Functional Loop (Comprehensions)

When we want to create a new list from an old one, we use **yield**. This is the "Functional Loop."

```
val names = List("alice", "bob", "charlie")
val capitalizedAdmins =
  for name <- names
    if name.startsWith("a") // The Guard: Filters the input
      yield name.capitalize // The Yield: Transforms the result
// Result: List("Alice")
```

While .map and .filter are concise for simple operations, the for-comprehension provides a more readable, declarative syntax for chaining transformations. It isn't a traditional loop; it is a powerful way to "sequence" operations—like pulling values out of a container and yielding a new result.

```
val numbers = List(1, 2, 3)
// For each value 'n' extracted from the numbers list,
// transform it and yield the result into a new list.
val doubled =
  for n <- numbers
    yield n * 2
// Result: List(2, 4, 6)
```

The syntax is deliberate:

- The **for** starts the expression.
- The **n <- numbers** (the generator) pulls each value out of the list one by one.
- The **yield** keyword tells Scala: "Take the result of this calculation and put it into a brand new list."

Why the "Mind Shift" Matters

It may help to think of Scala as a language designed with mathematical symmetry. In a traditional loop, you are managing the state of the computer. In a Scala comprehension or match statement, you are describing the relationship between your input and your output.

It's less about "doing" and more about "becoming."

So for comprehensions provide a much cleaner way to deal with iterations in may cases. The example below shows how it can clean up mutli-level iteration for example:

```
// The "Pyramid of Doom" with flatMap
val result = userOpt.flatMap { user =>
  addressOpt.flatMap { address =>
    getZipCode(address).map { zip =>
      s"${user.name} lives in $zip"
    }
  }
}
// The clean "for" version (Identical logic)
val result = for {
  user    <- userOpt
  address <- addressOpt
  zip     <- getZipCode(address)
} yield s"${user.name} lives in $zip"
```

As we mentioned, for comprehensions are also ideal for dealing with side-effects that don't just transform data.

A note on imperative style:

```
for {
  i <- 1 to 5
  if i != 3 // This is your "next"
} println(i)
```

Chapter 8

Exceptions, Catch & Throw

In a perfect world, databases never time out and users never type "potato" into an age field. In the real world, things go wrong. Scala assumes you know this, and it gives you a tiered defense system to handle the chaos.

The Traditional Guard: Try/Catch

If you've used Java or C#, the try/catch block will feel like a warm, familiar blanket. However, Scala gives it a "matching" upgrade. Instead of separate catch blocks for every type, you use a single match-style block to sort through the wreckage.

```
try {
  // A risky Doobie database call
} catch {
  case ex: java.sql.SQLException if ex.getErrorCode == 101 =>
    println("Specific DB Error: Check your permissions.")
  case ex: Exception =>
    println(s"Generic Error: ${ex.getMessage}")
}
```

Here we see some matching already - we prioritize the handler for sql exceptions, but also have a catch-all afterwards to deal with anything else. Every exception has a message, along with a getter and printer method for the stack trace.

Raising Exceptions

You can throw exceptions exactly as you'd expect. In fact, using Case Classes to define custom errors makes them incredibly easy to read and create.

```
case class DatabaseException(msg: String, code: Int) extends Exception(msg)
def getUser(id: Int): String =
  if id < 0 then throw DatabaseException("Invalid ID", 400)
  else "Alice"
```

You can see here that we do a basic check on parameters, and **throw** an exception when things go awry.

We can also catch multiple exceptions on a single line with the `|` operator:

```
case e @ (_: IllegalArgumentException | _: NullPointerException) =>
    println(s"Validation error: ${e.getMessage}")
```

Defining Exceptions

Creating new exception types is pretty straightforward. We just choose another exception type and extend it:

```
// A simple custom exception
class UserNotFoundException(message: String) extends RuntimeException(message
// Usage
throw new UserNotFoundException("User with ID 123 not found")
```

Though using case classes actually makes our lives a bit easier because we can then pattern match on the contents:

```
case class DatabaseError(code: Int, detail: String)
  extends RuntimeException(s"Error $code: $detail")
// You can now match on the specific fields
try {
  throw DatabaseError(500, "Connection Timeout")
```

```
} catch {
  case DatabaseError(500, _) => println("The DB is down!")
  case DatabaseError(code, _) => println(s"Other DB error: $code")
}
```

The Functional Guard: The Try Container

While try/catch works, it has a downside: it "interrupts" the flow of your program. In modern Scala, we prefer the Try container. Instead of a crash, it returns a "box" that contains either a Success or a Failure.

```
import scala.util.{Try, Success, Failure}
val result: Try[Int] = Try("123".toInt)
result match
  case Success(num) => println(s"Value is: $num")
  case Failure(ex) => println(s"Conversion failed: ${ex.getMessage}")
```

This is the "Pragmatic" way: wrap the dangerous code in the Try operator, and we can more quickly match on Success / Failure, allowing Failure to further handle the various exceptions if need be.

By returning a Try, you tell the next developer: "This might fail, and the compiler is going to make sure you deal with that possibility."

Either: Choosing a Side

The Either container is the preferred way to handle business logic errors. By convention, the **Left** side holds the error (because "left" is also a synonym for "sinister" or "wrong" in Latin roots) and the **Right** side holds the "right" (correct) result.

```
def divide(a: Int, b: Int): Either[String, Int] =
  if b == 0 then Left("Cannot divide by zero!")
  else Right(a / b)
val outcome = divide(10, 0) // Result: Left("Cannot divide by zero!")
```

System Hygiene: NonFatal and Finally

Not all errors are created equal. If your program runs out of memory (OutOfMemoryError), there is usually nothing you can do—the app is going

to crash. Scala provides the NonFatal helper to help you catch the "recoverable" stuff while letting the catastrophic stuff through.

```
import scala.util.control.NonFatal
try {
  // Risky System Access
} catch {
  case NonFatal(ex) => println(s"Recoverable error: $ex")
  // OutOfMemoryError will bypass this and crash the app safely
}
```

The finally Guarantee

When you open a file or a database connection, you must close it, regardless of what happens in between. The finally block is your iron-clad guarantee.

```
val source = scala.io.Source.fromFile("renaissance.txt")
try
  println(source.mkString)
finally
  source.close() // This runs NO MATTER WHAT
  println("Resource safely closed.")
```

Chapter 9

Traits

If Classes are the blueprints for your data, Traits are the blueprints for your behavior. In earlier versions of Scala, Traits were often seen as simple "Interfaces." In modern Scala, they have evolved into powerful Mixins that can carry both logic and state. They give you 2 major benefits:

1. Traits provide a namespace and prevent name clashes
2. Traits implement the mixin facility

The Anatomy of a Trait

As we write code, we will likely start to recognize chunks of code that can be reused. It helps to break this into different files that can be shared around the program. Because we are object oriented, this will likely end up contained in classes.

There are absolutely times where this stuff can't be naturally organized into classes however. For example, if 2 workers are developing their own libraries, they may end up accidentally using the same method names, or even the same class names. This is a pretty well-known problem, and the answer is namespaces – many languages use these – it's a well known pattern.

We can define a companion object on traits that allow us to scope methods to them:

```
trait User {
  def name: String // Instance method
```

```
}
object User {
  // This acts like a "static" method
  def anonymous: User = new User { val name = "Guest" }
  def fromString(name: String): User = {
    val n = name
    new User { val nme = n; def name = nme }
  }
}
// Usage:
val guest = User.anonymous
val biff = User.fromString(‘‘Biff’’)
```

More than that, however, traits significantly reduce the need for multiple inheritance.

When you look at a trait itself (without a companion object), it's not something that you can use on it's own. Instead, you mark classes as *extending* traits (can be multiple) – what this means is effectively that you get the methods (and instance values) defined in the trait for free. They effectively behave as super classes. This is exciting because now we get new methods we can call on the instance itself, as well as *within* methods of the instance.

```
trait HasNameTag {
  def prefix: String = "ID:" // A constant/default value

  // A method defined in the trait
  def formatName(name: String): String = s"[$prefix] $name"
}

class Robot(val model: String) extends HasNameTag {

  // 1. Internal Use
  // The class uses 'formatName' from the trait inside its own method
  def greet(): Unit = {
    val formatted = formatName(model)
    println(s"Hello, I am $formatted")
  }
```

```
}

val bot = new Robot("RX-9000")
// 2. External Use (on the instance)
// We can call the trait method directly on the bot object
val label = bot.formatName("Maintenance")
bot.greet()             // Prints: Hello, I am [ID:] RX-9000
println(s"Task: $label") // Prints: Task: [ID:] Maintenance
```

A Trait can define **abstract** methods (which have no body) and **concrete** methods (which do). When a class "mixes in" a Trait, it inherits all that functionality for free.

```
trait Logger:
  def log(msg: String): Unit // Abstract: You must define this later
  def info(msg: String) = log(s"[INFO] $msg") // Concrete: This logic is shar
```

We "mix in" traits using the *extends* keyword. If you want to add more than one, you simply use a comma.

```
class GenericService extends Logger:
  def log(msg: String) = println(msg) // We provide the 'how' for the log
```

Traits with Parameters

In Scala 3, Traits can now take parameters just like classes. This allows a Trait to make explicit assumptions about the data it needs to function.

```
trait DatabaseConfig(val dbName: String)
class DoobieRepository(name: String) extends DatabaseConfig(name):
  def connect() = println(s"Connecting to $dbName")
```

Self-Types: The "Only for Friends" Guard

Sometimes you want to write a Trait that only works if it's mixed into a specific type of class. This is called a Self-Type. It's like saying: "I am a MetricCollector, but I only know how to work when I'm part of a WebService."

```
class WebService:
  val apiKey = "SECRET_123"
trait MetricCollector:
  self: WebService => // This trait now 'thinks' it IS a WebService
  def logUsage() =
    // It has direct access to the WebService's apiKey!
    println(s"Sending metrics for API Key: $apiKey")
// This works because ProductionService extends WebService
class ProductionService extends WebService, MetricCollector
```

Sealed Traits: The Closed Door

A **Sealed Trait** is a powerful safety feature. When you mark a trait as sealed, you are telling the compiler: "Every single class that implements this trait must be in this exact same file."

This is the secret sauce for **Pattern Matching**. If the compiler knows every possible implementation of a trait, it can warn you if your match statement forgot one!

Organizing the World: Packages and Imports

As your application grows, you'll need a way to organize your traits and classes. Scala uses Packages to group related code together.

```
package io.startup.renaissance
case class User(name: String)
```

The Pragmatic Import

Scala's import system is incredibly flexible, allowing you to clean up your code and avoid naming "collisions" (like having two different Date classes).

- **The Wildcard:** *import scala.util.control.** imports everything in that package.
- **The Rename:** *import java.util.{Date => JDate}* lets you use JDate in your code to avoid confusion with other Date types.
- **The Selective:** *import sttp.client3.{basicRequest, UriContext}* only brings in exactly what you need.

Chapter 10

Basic IO

At some point, your beautiful Scala logic needs to interact with the cold, hard reality of the file system. Because Scala runs on the JVM, you have access to the ultra-reliable Java NIO (New I/O) libraries, but Scala provides some "syntactic sugar" to make these interactions feel much more natural.

Common Ops

The most common way to read from / write to standard i/o in scala is with the StdIn

```
// uses implicit StdOut
println("Hello, World!")
print("No newline here... ")
printf("Age: %d, Name: %s", 25, "Alice")
import scala.io.StdIn
println("Enter your name:")
val name = StdIn.readLine() // Returns a String
println("Enter your age:")
val age = StdIn.readInt()    // Returns an Int (throws exception if not a num
```

Reading Files: The Safe Way

The idiomatic way to deal with file reading & writing is with the scala.io.Source library.

The most common mistake in I/O is opening a file and forgetting to close it. In Scala, we use the .using pattern (or manual resource management) to ensure that once we are done reading, the file handle is snapped shut immediately.

```
import scala.io.Source
import scala.util.Using
// 'Using' is a manager: it opens the resource and closes it
// automatically even if your code crashes in the middle.
val lines: Try[List[String]] = Using(Source.fromFile("config.txt")) { source
  source.getLines().toList
}
lines match
  case Success(content) => println(s"Read ${content.size} lines.")
  case Failure(e) => println(s"Failed to read file: ${e.getMessage}")
```

By wrapping our file access in Using, we convert a risky operation into a Try container (which we learned about in Chapter 8). It's safe, predictable, and clean.

Writing Files: Leveraging the JVM

For writing data, we often lean on the robust java.nio libraries. While they look a bit more "Java-ish," they are incredibly fast and handle character encoding (like UTF-8) with precision.

```
import java.nio.file.{Files, Paths}
import java.nio.charset.StandardCharsets
val path = Paths.get("output.txt")
val data = "Hello, Renaissance!\nThis is Scala 3."
// Files.write is a 'one-shot' operation: it opens, writes, and closes.
Files.write(path, data.getBytes(StandardCharsets.UTF_8))
```

The Modern Approach: "Source" Shortcuts

If you just need to grab the entire contents of a file as a single string (common for config files or templates), Scala makes it a one-liner:

```
val content = Source.fromFile("banner.txt").mkString
```

A Note on Performance: Source.fromFile is great for simple tasks, but if you are processing a 10GB log file, you'll want to use source.getLines() which is lazy. It doesn't load the whole file into RAM; it just gives you one line at a time as you ask for it.

Creating Files

To create a new file, we simply use the default "write" method to fill it with content.

```
import java.nio.file.{Files, Paths}
import java.nio.charset.StandardCharsets
val path = Paths.get("data/config.txt")
val content = "key=value\nuser=admin"
// Creates the file and writes the string in one shot
Files.write(path, content.getBytes(StandardCharsets.UTF_8))
```

To append to a file, we use the same operation with some extra parameters:

```
import java.nio.file.{Files, Paths, StandardOpenOption}
val path = Paths.get("log.txt")
val logEntry = "New event happened\n"
Files.write(
  path,
  logEntry.getBytes,
  StandardOpenOption.CREATE, // Create if missing
  StandardOpenOption.APPEND  // Add to the end
)
```

Working with Paths

Instead of treating file locations as messy strings like "C:/users/docs/file.txt", we use the Paths and Path objects. This makes your code "cross-platform," meaning it will work on Windows, macOS, and Linux without you having to worry about backslashes versus forward slashes.

```
val dir = Paths.get("data", "logs")
val file = dir.resolve("app.log") // Automatically handles separators
println(s"Looking for file at: ${file.toAbsolutePath}")
```

Working with Different Chars/Bytes/etc

There are different options for reading from a file based on the chunk size you want your data in:

```
Files.readString(Path.of("file.txt")) // gets us the entire contents as a str
Source.fromFile("file.txt").getLines().foreach(println) // gets us contents l
val char = Source.fromFile("file.txt").next() allows us to read by character.
val bytes: Array[Byte] = Files.readAllBytes(path)
val buffer = new Array[Byte](1024); is.read(buffer) // allows us to read as a

import java.io.FileReader
val reader = new FileReader("massive_line.txt")
val buffer = new Array[Char](1024) // 1024 characters at a time
val charsRead = reader.read(buffer) // can optionally loop here, as charsRead
// buffer now contains the first 1024 characters
reader.close()
```

When it comes to writing, the same applies:

```
Files.writeString(Paths.get("output.txt"), "The whole story...")

// line by line
import java.io.PrintWriter
val pw = new PrintWriter("lines.txt")
pw.println("Line 1")
pw.println("Line 2")
pw.close()

import java.io.FileOutputStream
val os = new FileOutputStream("large_data.bin")
val chunk = new Array[Byte](4096) // 4KB increments
// Imagine a loop fetching data from a socket or another file
os.write(chunk, 0, chunk.length)
os.close()
```

Talking with Networks

Scala was built for the internet.

It can handle sockets & servers, be they TCP, UDP, whatever.

```
import java.net.Socket
import java.io.PrintWriter
import scala.io.Source
// 1. Open a TCP connection to port 80 (standard HTTP)
val socket = new Socket("google.com", 80)
val out = new PrintWriter(socket.getOutputStream, true)
// 2. Write the raw HTTP text
out.print("GET / HTTP/1.1\r\n") // Request Line
out.print("Host: google.com\r\n") // Required Header
out.print("Connection: close\r\n") // Tell server to hang up after sending
out.print("\r\n") // The mandatory empty line that ends the header section
out.flush()
// 3. Read the raw response
val response = Source.fromInputStream(socket.getInputStream).mkString
println(response)
socket.close()
```

But of course, that's not where most people live. Good thing the starndard higher level protocols are also supported (like HTTP et al).

```
import sttp.client3._
val request = basicRequest
  .get(uri"https://google.com")
  .header("User-Agent", "Scala-App-1.0")
  // sttp handles the \r\n and Host headers automatically
```

We can go even further and write straight to a file:

```
import sttp.client3._
import java.io.File
// This tells sttp to stream the response directly to a file in chunks
val request = basicRequest
  .get(uri"https://example.com/huge-report.csv")
```

```
  .response(asFile(new File("report.csv")))
val backend = HttpClientSyncBackend()
request.send(backend)
```

Of course, there's no error handling here, redirects, or really anything useful, but you get the idea.

Chapter 11

Threads & Processes

Scala still does enable multithreading via threads (concurrent executions of code, but still sharing the same process space and using it to cooperate) as well as processes themselves (each with its own memory space).

Multithreading

Often the way to do multiple things at once is via threading. These are all still within the same process space. And because Scala lives on the JVM, you can manage raw threads manually. It looks exactly like the Java you might remember from years ago:

```
val thread = new Thread(() => println("Running on a raw thread..."))
thread.start()
```

But manual threading is like trying to manage a busy kitchen by shouting at every individual cook. There are all sorts of things that can go wrong, in terms of prioritization, deadlock, waiting on long running system calls, trying to take advantage of multi-processor, starvation, etc. It's stressful and prone to accidents. However, the benefits of concurrency now are undeniable.

In modern Scala, a magical approach was devised that meant we don't manage threads, but get the benefits of multi-threading – **Futures**.

The Future: A Receipt for Your Result

A *Future* is a placeholder for a value that hasn't arrived yet. Think of it like a buzzer at a restaurant: you hold the buzzer (the Future), and eventually, it vibrates when your food (the Result) is ready.

```
import scala.concurrent.Future
import scala.concurrent.ExecutionContext.Implicits.global // The "Kitchen Sta
import scala.util.{Success, Failure}
val eventualData = Future {
  // This block runs on a background thread
  Thread.sleep(1000)
  "The secret ingredient"
}
eventualData.onComplete {
  case Success(value) => println(s"Order up: $value")
  case Failure(e) => println(s"The kitchen crashed: ${e.getMessage}")
}
```

Two key things to remember:

- **ExecutionContext**: This is the "Thread Pool." It's a group of worker threads waiting to pick up your tasks.
- **Eagerness**: Futures are eager. The moment you define one, it starts running in the background.

So we create a future with the Future(...) call. This *immediately* starts running, but in the background, so execution progresses to the next statement without waiting.

We can see on the next line, we define a "onComplete" handler for what to do when the Future finishes.

Waiting (The "Break Glass" Option)

Usually, we want our code to be "non-blocking," meaning it never stops to wait. But sometimes, especially in a small script or a console app, you just need the result now.

```
import scala.concurrent.Await
import scala.concurrent.duration.*
val eventualData = Future {
  // This block runs on a background thread
  Thread.sleep(1000)
  "The secret ingredient"
}
// We pause the current thread for up to 5 seconds
val result = Await.result(eventualData, 5.seconds)
println(s"I waited, and I got: $result")
```

So the Await will try to allow the Future to complete and get the data. But we don't want to wait forever, so we also give a timeout value.

In the case that something goes wrong, like an exception being raised within the Future code, we can use the recover clause to deal with it:

```
val handled = eventualData.recover {
  case e: ArithmeticException => 0
  case _ => -1 // Generic fallback
}
```

The "For" Pipeline: Stringing Futures Together

The real magic happens when you have multiple async tasks that depend on each other. Instead of "Callback Hell" (nesting onComplete inside onComplete), Scala uses the for-comprehension to create a clean, readable pipeline.

```
val result = for {
  user <- fetchUserFromApi(id) // Start task 1
  dbStatus <- saveToDatabase(user) // Once task 1 finishes, start task 2
} yield {
  s"Successfully saved ${user.name}"
}
```

A Note on Mutual Exclusion

In more traditional languages, mutual exclusion was often achieved with Mutexes, Locks, and other syncronization mechanisms. Scala allows this

via the JVM, but encourages strongly for developers to design around it:

Firstly, the immutable nature of default data goes a long way to avoid problems that way. Secondly, usage of Atomic data types for simple flags or values to avoid read/write race conditions. Pekko and other libraries are designed around *message-passing* over direct method invocation, again to allow for single thread code exection at a time.

Queues

Queues also enable async communication, whether within process or over the network. FS2 Streams and Channels are the go-to here.

Talking to the OS: The System Shell

Sometimes the best tool for the job isn't a Scala library, but a command that already exists on your machine. Scala makes the system shell feel like a native part of the language using the sys.process package.

By adding a simple ! or !! to a string, you can execute it as a shell command:

```
import sys.process.*
// 1. Just run it (prints to console, returns the exit code)
"ls -al".!
// 2. Capture the output (returns a String)
val myIp = "curl -s https://api.ipify.org".!!
// 3. The "Bash" Mode: Pipelining
// You can pipe commands together exactly like a shell script
val userCount = ("cat names.txt" #| "grep Alice" #| "wc -l").!!
println(s"Found $userCount instances of Alice.")
```

The simple *.!* and *.!!* methods make Scala an incredible language for writing system scripts that would usually be written in Bash or Python.

Multiple Processes

As mentioned, we can turn a string into a system command - process with a simple .!

```
import scala.sys.process._
// 1. Execute and get the exit code (0 usually means success)
val exitCode: Int = "ls -al".!
// 2. Execute and capture the output as a String
val output: String = "whoami".!!
println(s"Current user: ${output.trim}")
```

If we need system variables set in order to run, we can do that with the Process class.

Chapter 12

Testing

Testing is a must have in any modern language. It has a direct link to quality. It gives you constant feedback as you write, letting you know if you're headed in the right direction. It is a universally good thing.

Testing in Scala is primarily handled by ScalaTest. It is a massive, flexible framework that supports many styles, but the most "Pragmatic" choice is AnyWordSpec. It allows you to write tests that read like English sentences.

The WordSpec Style: Testing Behavior

Instead of naming a method test_price_calculation_with_tax, we describe the Subject and its Behavior.

```
import org.scalatest.wordspec.AnyWordSpec
import org.scalatest.matchers.should.Matchers
class ProductServiceSpec extends AnyWordSpec with Matchers:
  "A ProductService" should {
    "calculate tax correctly" in {
      val price = 100.0
      val tax = PriceCalculator.calculateTax(price)
      tax shouldBe 113.0 // This is the "Matchers" DSL
    }
    "return an error for negative prices" in {
      val result = PriceCalculator.calculateTax(-1)
      result shouldBe a [Left[_, _]] // Checking the "shape" of the data
```

```
    }
  }
```

Here the Subject is the ProductService, and the behaviour is price calculation. By mixing in Matchers, we get access to shouldBe. It makes assertions feel less like a math equation and more like a conversation.

Testing the Future

Testing asynchronous code (like our Futures from Chapter 11) is notoriously difficult in other languages, often involving "sleep" timers that make tests slow and flaky. ScalaTest handles this elegantly with the ScalaFutures trait.

```
import org.scalatest.concurrent.ScalaFutures
class ApiSpec extends AnyWordSpec with ScalaFutures:
  "The API" should {
    "fetch data eventually" in {
      val eventualData = mySttpClient.getData() // Returns a Future
      // 'whenReady' handles the waiting for you
      whenReady(eventualData) { data =>
        data.name shouldBe "Renaissance"
      }
    }
  }
```

Structuring tests this way gives us a place to put individual tests, providing a structure (around services). We also get a few options for how to run tests:

```
Run all tests:
sbt test
Run a specific test suite:
sbt "testOnly MyFirstTest"
Continuous testing (Watch mode): This is the "pro" way. sbt will watch your f
sbt ~test
```

The “shouldBe” operation is generally a test for equality, but can also be used with “empty” on collection types, or even on types (eg. data.name shouldBe a [String])

Unlike the assert operation inherited from Java, shouldBe will return a more gracious message on failure.

Property-Based Testing: The Stress Test

Sometimes, writing individual test cases isn't enough. What if a bug only appears when a string is empty, or a number is exactly Int.MaxValue?

Property-based testing (often using a tool called ScalaCheck alongside ScalaTest) allows you to define a "property" that must always be true, and the library will bombard your code with random data to try and break it.

```
// Conceptual Example:
"The Calculator" should {
  "always return a positive number when squaring" in {
    forAll { (n: Int) =>
      Calculator.square(n) should be >= 0
    }
  }
}
```

Integration: Doobie and sttp

When you move into full application testing, your strategy will rely on the libraries we've used throughout this tour:

- **sttp**: Provides a "Testing Backend" that lets you mock API responses without actually hitting the network.
- **Doobie**: Provides a *doobie-scalatest* module that can actually check your SQL strings against your database schema at compile time to ensure your queries are valid.

We can add before {} and after {} clasues to our tests for setup & teardown code to run before and after each test.

The standard for tests is one test for every class, named with a "Spec" postfix.

We do also encourage unit and integration tests. Unit tests go in src/test/scala. Integration tests go in src/it/scala.

Generally, we make use of traits to create shared resources the tests can use.

Part II

Scala Surroundings

Chapter 13

Scala and Its World

scala-cli & command-line options

Before you even run a single line of your code, you have to talk to the scala-cli tool. This tool is responsible for fetching your libraries, choosing your Scala version, and packaging your app.

The Essentials: Running and Checking

The most basic commands allow you to check your environment and run your scripts instantly.

- **Check Version**: *scala-cli version* (Tells you the tool version).
- **Check Scala Version**: *scala-cli . --scala-version* (Tells you which Scala version is currently active).
- **Run a Script**: *scala-cli MyScript.scala* (Compiles and runs in one go).

Dependency Management on the Fly

One of the best "Pragmatic" features of scala-cli is that you don't need a complex build file to test a library. You can "require" it directly from the command line.

```
# Run a script while pulling in the sttp library instantly
scala-cli MyScript.scala --dep com.softwaremill.sttp.client3::core:3.9.0
```

Log Levels and Debugging

If your build is failing or a library isn't downloading, you need to see what's happening under the hood.

- **Verbose Mode**: *-v or -vv or -vvv*. Adding more vs increases the "chatter" from the tool.
- **Log Level**: *--log-level debug* (Gives you the surgical details of the compilation process).

Directing the Output (Packaging)

When you are ready to move from "scripting" to "shipping," scala-cli can package your code into different formats.

- **Create a Lightweight Runner**: *scala-cli package . -o my-app* (Creates a launcher script).
- **Create an Assembly (Fat Jar)**: *scala-cli package . --assembly -o app.jar* (Everything in one file).
- **Create a Native Image**: *scala-cli package . --native-image -o fast-app* (Uses GraalVM for instant startup).

Using Directives (The "In-File" Args)

While you can pass all these as command-line arguments, it's much more "Pragmatic" to put them at the top of your file using Using Directives. This ensures that anyone who runs your file has the exact same settings.

```
//> using scala "3.3.1"
//> using dep "com.softwaremill.sttp.client3::core:3.9.0"
//> using dep "com.lihaoyi::pprint:0.8.1"
@main def hello = pprint.log("Directives make this script self-contained!")
```

Common Developer Flags

--watch: The "Developer's Best Friend." It keeps the process alive and reruns your code every time you save the file.

--jvm: Allows you to specify a specific JVM version (e.g., *--jvm 17*).

--repl: Opens an interactive shell with all your libraries and code preloaded.

your code & the command-line

A great tool should be configurable from the outside. Whether you are passing a file path, a database URL, or a debug flag, your application needs to parse the "intent" of the user from the terminal.

The @main Entry Point

In the old days, you had to define a specific main method inside an object. In Scala 3, you simply annotate a method with @main. Scala automatically handles the conversion of string arguments into the data types you specify.

```
@main def runScanner(path: String, iterations: Int): Unit =
  println(s"Scanning $path for $iterations cycles...")
```

If you run this from your terminal: *scala runScanner.scala /var/logs 5*

Scala sees that iterations should be an Int and performs the conversion for you. If the user passes "five" instead of "5", Scala will provide a helpful error message automatically.

Handling Multiple Arguments (Varargs)

Sometimes you want to accept an unknown number of inputs—like a list of files to process. We use the "Splat" syntax (*) we learned in Chapter 6.

```
@main def bulkDelete(files: String*): Unit =
  println(s"Deleting ${files.length} files...")
  files.foreach(f => println(s"Removing $f"))
```

Optional Arguments and Defaults

Not every argument should be mandatory. By providing a default value in your method signature, you make that command-line flag optional.

```
@main def startServer(port: Int = 8080, verbose: Boolean = false): Unit =
  if verbose then println("Verbose mode active.")
  println(s"Server starting on port $port")
```

Raw Access: The args Array

If you need total control—perhaps to build a complex CLI with nested commands—you can still access the raw array of strings exactly as they were typed.

```
@main def rawHandler(args: String*): Unit =
  args.toList match
    case "start" :: path :: Nil => println(s"Starting at $path")
    case "stop" :: Nil => println("Stopping...")
    case _ => println("Usage: start <path> | stop")
```

This uses the **Pattern Matching** on Lists that we covered in Chapter 7, turning the command line into a clean, readable dispatch table.

Environment Variables

There are a few ways to grab enironment values, depending on what exactly you need:

scala.util.Properties - best for quick OS-based logic:

```
import scala.util.Properties
println(s"OS Name: ${Properties.osName}")
println(s"Java Home: ${Properties.javaHome}")
println(s"Scala Version: ${Properties.versionNumberString}")
```

System.getProperty - best for JVM startup flags:

```
// CPU Architecture (e.g., x86_64, aarch64)
val arch = System.getProperty("os.arch")
// Library Directory (Where the OS looks for .so, .dll, or .dylib files)
val libPath = System.getProperty("java.library.path")
// Temporary Directory (Good for File IO chunks we discussed)
```

```
val tmpDir = System.getProperty("java.io.tmpdir")
// User's Working Directory
val userDir = System.getProperty("user.dir")
println(s"Arch: $arch")
println(s"Lib Dirs: $libPath")
```

General system info - best for Shell/OS environment, like passwords, docker setting, config:

```
import scala.jdk.CollectionConverters._
val allProps = System.getProperties.asScala
allProps.foreach { case (k, v) => println(s"$k -> $v") }
```

Best Practices for CLI Tools

- **Fail Fast**: Use the types (like Int or Boolean) in your @main signature so the program crashes with a usage message before any heavy logic starts.
- **Use help**: While Scala 3 provides basic errors, for complex tools, the community often reaches for libraries like **MainArgs** or **Decline** to generate "help" menus automatically.
- **Environment Variables**: For sensitive data (like our **Doobie** database passwords), never use command-line args. Use *sys.env.get("DB_PASSWORD")* instead to keep them out of the process list.

Dependencies

When you use a *//> using dep* directive or a *--dep* flag, scala-cli isn't just wishing those libraries into existence; it's using a specialized engine called Coursier.

Where do the modules go?

Scala-cli doesn't litter your project folder with a node_modules or *vendor* directory. Instead, it uses a **Global Central Cache**.

1. **The Fetch**: When you run your code, scala-cli calls Coursier. It looks at "Maven Central" (the massive, global library of JVM jars) and downloads the specific version you requested.

2. **The Cache**: The JAR files are stored in a hidden folder in your home directory:
   - **macOS/Linux**: *~/Library/Caches/Coursier/v1* or *~/.cache/coursier*
   - **Windows**: *%LOCALAPPDATA%\Coursier\Cache\v1*
3. **The Link**: When you compile, scala-cli simply "points" your project to these JARs in the cache.

Chapter 14

Interactive Scala

The REPL: Your Experimental Sandbox

Sometimes you don't want to write a whole script just to see how a Regex behaves or how a List transforms. You want an immediate answer.

By running *scala-cli repl .*, you launch an interactive session that is context-aware. It doesn't just give you a blank slate; it loads every dependency and every class defined in your current project. All in an environment that gives you immediate feedback:

```
# Launch the REPL with your project's superpowers from DOS/shell prompt
scala-cli repl .
```

The repl of course comes with several switches to modify it's behaviour:

Switch	Purpose	scala-cli Examp
--dep	Adds libraries (replaces -cp)	scala-cli repl --dep com.softwaremill
--deprecation	Shows detailed warnings	scala-cli run . --depre
--feature	Enables advanced language features	scala-cli run . --fea
-i	Pre-loads a script into the REPL	scala-cli repl -i setup
-e	Executes a string and exits	scala-cli run -e "printl

As with all CLIs, functions, constants, and general context is available going forward.

This is the ideal place to test out classes & functions you have built to see how they handle various scenarios.

The most common way to dynamically pull in updated code is the :load command. It reads a .scala file and executes every line as if you typed it into the REPL manually.

```
scala> :load MyScript.scala
// All classes, objects, and variables in MyScript.scala are now updated in y
```

However, due to scala's sophistication, we can tell the scala-cli to dynamically reload any libraries it has loaded whenever they change:

```
scala-cli repl . --watch
```

You can also simply redefine things on the fly, such as classes, in the REPL:

```
object Config { val url = "http://new.com" } // replaces existing Config valu
```

Tab Completion

The repl has a very sophisticated tab completion feature. Simply partially type a class/funtion/variable/whatever, and press tab to get a list of possible completions. Saves on type, and also helps to explore classes and methods.

Inside the REPL, you can:

Test Snippets: Paste in a complex match statement to see if it catches your edge cases.

Inspect Types: Use :type myVariable to see exactly what the compiler thinks you've created.

Tab-Complete: Hit Tab to see every method available on an object (perfect for exploring the 200+ methods on a String).

Quick Imports: Just like in your code, you can import sttp.client3.* and start making live API calls from the command line.

It is the ultimate "Pragmatic" tool for debugging: move from "I think this works" to "I know this works" in seconds.

Configuration

There are several frameworks available for scala that provide customized REPLs with their own initialization code, etc. However, we're going to stick to the OG here.

For the standard REPL we have been discussion, you can provide "preloaded" code that runs whenever a REPL is opened. For example, if we write some initial code in **init.scala**

```
import sttp.client3._
import doobie._
import doobie.implicits._
import cats.effect.IO
// Pre-define your standard backend and database connection
val backend = HttpClientSyncBackend()
val xa = Transactor.fromDriverManager[IO](...)
println("Environment loaded: sttp and doobie ready.")
```

We can load it with:

```
scala-cli repl --dep ... -i init.scala
```

In fact, we can define all sorts of helper methods and classes that we can then use in the repl console with:

```
//> using dep com.softwaremill.sttp.client3::core:3.9.0
//> using dep org.tpolecat::doobie-core:1.1.0
import sttp.client3._
import doobie._
import doobie.implicits._
// 1. Define a helper Class for API calls
case class Api(token: String) {
  val base = basicRequest.auth.bearer(token)
  def getUser(id: Int) = base.get(uri"https://api.com/users/$id")
}
// 2. Define a helper Method for DB status
def checkDb(xa: Transactor[cats.effect.IO]): Unit = {
  import cats.effect.unsafe.implicits.global
```

```
  val result = sql"SELECT 1".query[Int].unique.transact(xa).unsafeRunSync()
  println(s"Database Status: $result (Success)")
}
println("--- Dev Environment Loaded ---")
println("Try: val api = Api(\"my-secret\")")
```

Configuration

To see current settings: :settings

To change a setting: :settings -deprecation or :settings -Xlint

To see the "Power" mode (internal compiler debug): :power

Note: Power mode is for advanced users; it gives you access to the compiler's internal symbol table.

You can use the import $dep syntax to pull in a library on the fly. You don't need to restart.

```
// Pull in the sttp Circe (JSON) module mid-session
import $dep.`com.softwaremill.sttp.client3::circe:3.9.0`
// Now you can immediately use it
import sttp.client3.circe._
```

We can also update log levels:

```
import org.slf4j.LoggerFactory
import ch.qos.logback.classic.{Level, Logger}
// Get the root logger and silence it, or set to DEBUG to see SQL/HTTP traces
val root = LoggerFactory.getLogger(org.slf4j.Logger.ROOT_LOGGER_NAME).asInsta
root.setLevel(Level.DEBUG)
// Now Doobie will likely print every SQL statement it executes!
```

Finally, if you're unsure what's currently available in your session:

:imports: Lists all the import statements currently active in your session.

:type: Check the type of any variable to see if a module's implicit conversion has "taken over" your object.

:javap: If you want to see the "real" Java methods on a class you just loaded.

Commands

There are commands to interact with the REPL beyond standard classes & methods

Command	Action	Key
:quit (or :q)	Exits the REPL session entirely.	Ctrl+D (Uni
:help	Displays a list of all available colon commands.	
:reset	Forgets all defined variables and classes but stays in the REPL.	
:history	Shows the last few lines you typed.	Up/Dow

In addition to:

:load <path>: Pulls in an external file.

:save <path>: This is a "hidden gem." It saves your entire current REPL session history to a file so you can turn your experiments into a script later.

import $dep.<coordinates>: Dynamically adds a library (like we saw for adding JSON support to sttp).

import $file.<path>: Similar to :load, but treats the file as a module you can reference.

Some Extras

If you want more informative outputs as you run scripts, use:

```
scala> :settings -print-types
```

Chapter 15

Documenting Scala

Good documentation in Scala lives directly above your methods and classes. By using a specific comment syntax (/** ... */), you allow the scaladoc tool to extract your thoughts into a professional documentation suite.

The Scaladoc Syntax

Scaladoc uses Markdown-style formatting inside the comments. This means you can use backticks for code and asterisks for bold text without learning a complex new language.

```
/** Represents a company in the Renaissance system.
*
* @param name The legal name of the entity
* @param revenue The current MRR in USD
*/
case class Company(name: String, revenue: Int)
```

Linking and Referencing

One of the most powerful features of Scaladoc is the ability to link to other parts of your code. Instead of telling someone to "see the Product class," you can link to it directly using double square brackets. The tool will verify that the class actually exists.

```
/**
* Processes payments for a [[Company]].
* * If the transaction fails, it returns a [[scala.util.Failure]].
*/
def processPayment(c: Company): Unit = ???
```

But in fact you can link to arbitrary hyperlinks as well with [Label](URL) syntax

```
/** * Fetches a user from the DB.
 * * @param id the unique user identifier
 * @return a [[doobie.ConnectionIO]] containing the user
 * @note Requires a [[doobie.util.transactor.Transactor]] to execute.
 * @see [[https://tpolecat.github.io/doobie/ Doobie Documentation]]
 */
def findUser(id: Int): ConnectionIO[User] = ???
```

We can also create lists, which comes in handy for readability:

```
/**
 * To perform a Doobie transaction:
 * 1. Define your query using the ‘sql‘ interpolator.
 * 2. Map the result to a case class.
 * 3. Use a [[doobie.util.transactor.Transactor]] to run it.
 * * Key features:
 * - Type-safe SQL
 * - Automatic resource management
 * - Streaming support via FS2
 */
```

Headings

Markdown headings are full supported, helping with readability in the generated docs:

```
/**
 * # User Management Service
 * This service handles all database interactions for Users.
```

```
 * * ## Database Operations
 * These methods use [[doobie]] to execute SQL.
 * * ### Read Operations
 * Methods for fetching data.
 */
object UserService { ... }
```

Common Tags

To keep your docs organized, use the standard "at" tags. These create specific sections in the generated output:

- *@param*: Describes a method or constructor parameter.
- *@return*: Describes what the caller can expect back.
- *@throws*: Documents the exceptions that might be raised (see Chapter 8).
- *@note*: Adds a critical piece of information or a "Gotcha."
- *@see*: Points to external documentation or related classes.

Generating the Site

With *scala-cli*, you don't need to install a separate documentation engine. You can generate your entire documentation site with a single command:

```
scala-cli doc . -o ./my-docs
```

This will scan your project, resolve all your links, and spit out a folder containing a static website. You can open *index.html* in any browser to see your work.

Documenting with Directives

Just as we use //> *using* for dependencies, we can use them to configure how our documentation is built. You can set the project name, version, and even include a custom logo.

```
//> using docName "Renaissance App"
//> using docVersion "1.0.0"
```

The "Pragmatic" Minimum

You don't need to document every single private method. Focus your energy on:

1. **Public APIs**: Anything another developer (or you, six months from now) will call.
2. **The "Why"**: Code explains how; comments should explain why.
3. **Examples**: Using the *@example* tag to show a snippet of the method in action is often more helpful than three paragraphs of text.

Chapter 16

Package Management with Scala

Once your code works on your machine, the next challenge is getting it to work on someone else's. This involves two things: managing your internal code structure (Packages) and bundling your application for the world (Publishing).

Scala packages are a standardized packaging and installation framework for libraries and applications that make it easy to locate, install, uninstall, and upgrade your packages. It provides:

- a standard package format
- a central repository for hosting packages of this format
- installation & management of multiple installed versions of the same library
- end-user tools for querying, installing, uninstalling & otherwise managing these packages

In the scala world, devs bundle apps. Most commonly, they bundle them into *jars (just like Java). If the any library you request is missing, when you run your projects, scala-cli first checks your local cache (often somewhere like ~/.cache/coursier or ~/.ivy2), and if the library you asked for isn't there it reaches out and downloads the JAR and POM (describing what the library itself needs) from *Maven Central*. It uses this same approach recursively on the packages required by each package until all the needed code is on the local machine.

In the scala-cli world, devs package their libraries and applications into JARs with POMs. These conform to a standardized format, and the scala-cli tool provides a great way for manipulating them.

The Internal Structure: Packages

As we touched on in Chapter 9, packages are the "folders" of your logic. They prevent name collisions and organize your code into logical domains.

```
package io.startup.renaissance.auth
class Authenticator:
  def login() = ???
```

The **Pragmatic** rule: Your package names should usually mirror your folder structure (e.g., src/auth/Authenticator.scala), making it easy for developers to find files just by looking at the import statements.

The project.scala File: The Source of Truth

Instead of scattering settings across multiple files, we centralize everything in project.scala. This is your manifest. It tells scala-cli who you are, what version you're on, and what you need.

```
//> using scala "3.3.1"
//> using dep "com.softwaremill.sttp.client3::core:3.9.0"
//> using dep "org.tpolecat::doobie-core:1.0.0-RC4"
// Publishing metadata
//> using publish.name "renaissance-core"
//> using publish.organization "io.startup"
//> using publish.version "0.1.0"
```

This will automatically ensure these libraries have what they need and are themselves installed the next time you run.

Alternatively, when running a scala script, you can add –dep arguments detailing the libraries required.

Documentation

When you build documentation for your application, the tool is smart – it will look at the //> using dep lines and try to link to official documentation for those libraries.

Creating your Own Libraries

So, we established that using external libraries with scala (from Maven Central) is powerful and easy. It would be nice if we could create our own to help structure our code and allow others to use them. Let's take a look at that.

Let's say our startup tracking app is in a nice, finished state. Wouldn't it be nice if we could make it available for others?

Package Layout

The nice thing about scala is that it is refreshingly flat and clear – there's not a lot you need to do to your project in order to be able to package it. For our project, here's what we might look like:

renaissance-project/
├── project.scala <-- Best practice: centralize your "using" directives here
└── main.scala <-- Your @main entry point

Our project.scala would contain "build" configuration code like:

```
//> using scala 3.x
//> using dep com.softwaremill.sttp.client3::core:3.9.11
//> using dep org.tpolecat::doobie-core:1.0.0-RC5
//> using dep org.postgresql:postgresql:42.7.2
```

The main.scala would create our application code, cleared of all the configration code.

If we wanted to put scala code in folders, the scala-cli will automatically detect those as well. We could add data models to a "/models" folder, for example, to keep things tidy.

On top of that. should you have resource files (If your Doobie setup requires raw .sql files or your sttp client needs a logback.xml for logging, for example), you should use a /resources folder, and place them in ther.

Once our files are structured properly, we could run

```
scala-cli package .
```

to grab every scala file (including in subfolders) and compile them together.

Creating an Executable (Fat JARs)

The most common way to share a Scala app is a "Fat JAR" (or Assembly). This is a single file that contains your code **plus** every library it needs to run (like sttp and Doobie).

```
scala-cli package . --assembly -o renaissance.jar
```

Now, anyone with a JVM can run your app with: java -jar renaissance.jar.

Native Binaries: No JVM Required

For CLI tools where you want "instant-on" performance, you can package your app as a native binary. This uses GraalVM to compile your Scala code down to machine code (like C++ or Go).

```
scala-cli package . --native-image -o renaissance
```

This produces a file that runs natively on the target OS, with no Java installation required by the end-user.

Publishing Specifications

In our project.scala file, we can give details that get associated with the package you are publishing:

```
//> using publish.organization "io.github.renaissance"
//> using publish.name "renaissance-core"
```

```
//> using publish.version "0.1.0"
//> using publish.url "https://github.com/your-repo/renaissance"
//> using publish.license "Apache-2.0"
//> using publish.developer "YourName|YourEmail|https://your-website.com"
// Point to the modern Sonatype portal
//> using publish.repository "central-s01"
```

To publish publicly, to get yourself ready, you can run the following command to get setup:

```
scala-cli --power publish setup .
```

Adding Tests

Any good package will need to be sufficiently tested just out of good manners. Just like our source, the layout is very flexible.

Any file ending in the suffix .test.scala is automatically treated as a test source. Running:

```
scala-cli test .
```

will run every test.

Tests should either be kept next to the source logic they are testing, or in a tests/ subfolder.

Note that in project.scala, we will want to add:

```
// This is only downloaded/used when running 'scala-cli test'
//> using test.dep org.scalameta::munit::1.0.2
```

Which, as state, will only be used when testing. This states we are using MUnit, which is a modern, lightweight, and has great integration wil sttp's Future and IO responses.

Publishing to the World

If you've built a library that others should use, you'll want to "publish" it to a repository (like Maven Central or a private GitHub Packages repo).

To get set up, we will first need to do a few things:

- Create a Sonatype Account: Sign up at the Maven Central Portal.

- Claim your Namespace: You must prove you own the domain (e.g., io.github.yourusername or com.yourdomain). Sonatype will ask you to create a specific DNS TXT record or a public GitHub repo to verify ownership.

- Set up a gpg key:

```
# Generate your key
gpg --gen-key
# List it to find your Key ID
gpg --list-keys
# Send it to a public server so Maven can verify it
gpg --keyserver keyserver.ubuntu.com --send-keys YOUR_KEY_ID
```

You will also need to setup passwords:

```
scala-cli config publish.credentials central-s01 env:SONATYPE_USER env:SONATY
```

But when it comes to actually publishgin, scala-cli makes this surprisingly simple:

```
# This prepares your library for a repository
scala-cli publish .
```

Managing Versions with "Directives"

One of Scala's greatest strengths is "Typelevel" versioning. When you see a double colon :: in a dependency, it tells Scala to look for the version of the library that matches your specific Scala version.

- com.lihaoyi::pprint:0.8.1 $>$ "Get me the version of pprint built for Scala 3."

Chapter 17

Scala & the Web

Scala's web ecosystem is built for scale, but that "scale" doesn't mean "complexity." You can write your own servers in scala, but more interestingly you can serve up web logic with scala (using libraries to handle the protocol part).

In this chapter, we move beyond scripts and into the world of REST APIs and HTTP services. There are several options for this, and we can't possibly do it all justice. We will try to touch on some highlights, give you a lay of the land, and point you towards resources so you can explore further.

Let's start with the simple stuff - coming up with a http server with endpoint handlers.

The Tapir Philosophy: Endpoints as Values

In most frameworks, an endpoint is a function. In Scala, an endpoint is a `value`. This means you can describe your API (its inputs, outputs, and errors) once, and then use that description to generate a server, a client, and documentation.

```
import sttp.tapir.*
import sttp.tapir.json.zio.* // Define the "Shape" of your endpoint
val helloEndpoint = endpoint.get
  .in("hello")
  .in(query[String]("name"))
  .out(stringBody)
```

```
  .errorOut(stringBody)
```

Serving the API

Once the shape is defined, you simply "bind" logic to it. Because we are using `sttp` for HTTP, the transition from definition to execution is seamless.

```
val helloLogic = helloEndpoint.serverLogic { name =>
  if (name.isEmpty) Future.successful(Left("Name cannot be empty"))
  else Future.successful(Right(s"Hello, $name! Welcome to Renaissance."))
}
```

Return Code

Now, web applications often don't just return plain text when they are called. We have templates (often of HTML) that we like to perhaps put values into. pebble templates fill this function nicely.

We could define a template (often in resources/templates/home.html) like this:

```
<!DOCTYPE html>
<html>
<body>
    <h1>Welcome, {{ name }}!</h1>
    <p>Status: <strong>{{ status }}</strong></p>
    <ul>
        {% for item in items %}
            <li>{{ item }}</li>
        {% endfor %}
    </ul>
</body>
</html>
```

In our main scala code that responds to queries, we would use:

```
//> using scala 3.3.1
//> using dep io.pebbletemplates:pebble:3.2.2
```

```
//> using dep com.softwaremill.sttp.tapir::tapir-netty-server:1.10.0
import io.pebbletemplates.pebble.PebbleEngine
import java.io.StringWriter
import scala.jdk.CollectionConverters._
object TemplateEngine {
  val engine = new PebbleEngine.Builder().build()
  val compiledTemplate = engine.getTemplate("templates/home.html")
  def render(name: String, status: String, items: List[String]): String = {
    val writer = new StringWriter()
    // Pebble expects a Java Map for the context
    val context = Map(
      "name" -> name,
      "status" -> status,
      "items" -> items.asJava
    ).asJava
    compiledTemplate.evaluate(writer, context)
    writer.toString
  }
}
// Then in your Tapir server logic:
// val htmlResult = TemplateEngine.render("Gemini", "Online", List("sttp", "D
```

Which would take the template defined and mix into some nice dynamic markup.

Database the things

In fact, we could take the previous code and use a database query to return some dynamic results:

```
//> using scala 3.3.1
//> using dep com.softwaremill.sttp.tapir::tapir-netty-server:1.10.0
//> using dep io.pebbletemplates:pebble:3.2.2
//> using dep org.tpolecat::doobie-core:1.0.0-RC5
//> using dep org.postgresql:postgresql:42.7.2
import sttp.tapir._
import sttp.tapir.server.netty.NettyFutureServer
import io.pebbletemplates.pebble.PebbleEngine
```

```
import doobie._
import doobie.implicits._
import cats.effect.unsafe.implicits.global
import scala.concurrent.ExecutionContext.Implicits.global
import scala.concurrent.Future
import java.io.StringWriter
import scala.jdk.CollectionConverters._
object Main extends App {
  // 1. Setup Doobie (The Database)
  val xa = Transactor.fromDriverManager[cats.effect.IO](
    "org.postgresql.Driver", "jdbc:postgresql:renaissance", "user", "pass"
  )
  // 2. Setup Pebble (The Template Engine)
  val engine = new PebbleEngine.Builder().build()
  val compiledTemplate = engine.getTemplate("templates/home.html")
  // 3. Define the Web Endpoint
  val indexEndpoint = endpoint.get
    .in("welcome" / path[String]("name"))
    .out(htmlBody) // Tells Tapir to set Content-Type: text/html
    .handle { name =>
      // A. Fetch data from DB using Doobie
      val dbAction = sql"SELECT item_name FROM inventory".query[String].to[Li

      // B. Run DB action and render template
      val items = dbAction.unsafeRunSync() // Simple sync for this example

      val writer = new StringWriter()
      val context = Map("name" -> name, "items" -> items.asJava).asJava
      compiledTemplate.evaluate(writer, context)

      Future.successful(Right(writer.toString))
    }
  // 4. Run the Server
  println("Server online at http://localhost:8080/welcome/Gemini")
  NettyFutureServer().addEndpoint(indexEndpoint).start()
}
```

We will talk about this more below.

Query Parameters

Query parameters are tacked on to endpoints:

```
// Matches: /welcome?name=Gemini&age=25
val welcomeEndpoint = endpoint.get
  .in("welcome")
  .in(query[String]("name")) // Mandatory
  .in(query[Int]("age"))    // Mandatory (will error if not a number)
  .out(htmlBody)
  .handle { (name, age) =>
    // Tapir passes them as a tuple to your handler
    Future.successful(Right(TemplateEngine.render(name, age)))
  }
```

By default, these will be mandatory. However, we can also take optional parameters:

```
// Matches: /items or /items?category=books
val itemsEndpoint = endpoint.get
  .in("items")
  .in(query[Option[String]]("category"))
  .handle { categoryOpt =>
    // categoryOpt will be Some("books") or None
    val category = categoryOpt.getOrElse("all")
    // Use Doobie to fetch based on category...
    Future.successful(Right(renderItems(category)))
  }
```

The JSON Connection

Scala excels at Case Class to JSON conversion. By using a library like `ZIO-JSON`, you can send and receive complex objects without manual parsing.

```
case class CompanyStats(name: String, revenue: Int)
// Tapir handles JSON conversion based on your case class
val statsEndpoint = endpoint.get
  .in("stats" / path[String]("orgId"))
  .out(jsonBody[CompanyStats])
```

Connecting the Database: Doobie in the Web

A web app is only as good as its data. Following our project standard, we use `Doobie` to bridge the gap between our endpoints and our SQL store.

```
import doobie.*
import doobie.implicits.*
def getStats(orgId: String): ConnectionIO[CompanyStats] =
  sql"SELECT name, revenue FROM organizations WHERE id = $orgId"
    .query[CompanyStats]
    .unique
// In your server logic, you run the Doobie program:
val statsLogic = statsEndpoint.serverLogic { id =>
  getStats(id).transact(xa).map(Right(_))
}
```

Cookies

Cookies are a way for web apps to store state. They are useful to store session information, for example.

Reading cookie values is pretty straightforward:

```
val dashboardEndpoint = endpoint.get
  .in("dashboard")
  .in(cookie[Option[String]]("session_token")) // Optional cookie
  .handle {
    case Some(token) =>
      // Verify token in your Doobie database
      Future.successful(Right(s"Welcome back! Your token is $token"))
    case None =>
      Future.successful(Left("Please log in."))
  }
```

Writing cookie values is equally simple:

```
import sttp.model.headers.CookieValueWithMeta
val loginEndpoint = endpoint.post
  .in("login")
  .out(setCookie("session_token")) // This header will be sent to the browser
```

```
  .handle { _ =>
    val cookieValue = CookieValueWithMeta.safe("xyz-123-abc")
      .expires(java.time.Instant.now().plusSeconds(3600)) // 1 hour
      .httpOnly(true) // Security best practice
      .secure(true)   // Only sent over HTTPS

    Future.successful(Right(cookieValue))
  }
```

Sessions

Session info is really what we want – it powers the majority of web applicatins. Sessions emulate hashlike behaviour.

In our case ,we need to do a couple of things manually to support sessions.

First, we'll need to create a table:

```
CREATE TABLE sessions (
    id VARCHAR PRIMARY KEY,
    user_id INT NOT NULL,
    data JSONB, -- You can store JSON here
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Then, define exactly what it is we want to store. In our case, we will just store a few details about the session:

```
import io.circe.generic.auto._
import io.circe.syntax._
import io.circe.parser._
case class UserSession(
  username: String,
  isAdmin: Boolean,
  lastLogin: String,
  theme: String
)
```

Now, on login, store the data to the db:

```
import java.util.UUID
import sttp.model.headers.CookieValueWithMeta
val loginEndpoint = endpoint.post
  .in("login")
  .out(setCookie("session_id"))
  .handle { _ =>

    val sessionData = UserSession("alice", true, "2026-03-05", "dark")
    val sessionId = UUID.randomUUID().toString
    // Doobie turns the case class into a JSON string (using Circe)
    val action = sql"""
      INSERT INTO sessions (id, data)
      VALUES ($sessionId, ${sessionData.asJson.noSpaces})
    """.update.run.transact(xa)
  }
```

Now we can lookup the session data by session_id from the cookie and use it in response when we are logged in:

```
val profileEndpoint = endpoint.get
  .in("profile")
  .in(cookie[String]("session_id"))
  .handle { sessionId =>
    // 1. Fetch the JSON string from DB
    val jsonString = sql"SELECT data FROM sessions WHERE id = $sessionId"
      .query[String].unique.transact(xa).unsafeRunSync()
    // 2. Turn the JSON string back into our Scala case class
    decode[UserSession](jsonString) match {
      case Right(session) =>
        Future.successful(Right(s"Hello ${session.username}, you are using ${
      case Left(_) =>
        Future.successful(Left("Session corrupted"))
    }
  }
```

Automatic Documentation

Because your endpoints are values, Scala can look at them and generate an OpenAPI/Swagger UI automatically.

```
import sttp.tapir.swagger.bundle.SwaggerInterpreter
val swaggerEndpoints = SwaggerInterpreter()
  .fromEndpoints(List(helloEndpoint, statsEndpoint), "My API", "1.0")
```

Launching the Server

Describing an endpoint is one thing; actually listening for traffic on port 8080 is another. To stay "Pragmatic," we want a server that is fast, lightweight, and easy to configure. Using the `NettyFutureServer` interpreter, we can turn our Tapir logic into a running process.

The Main Entry Point

Following our Scala 3 standards, we use the `@main` annotation to create our server's entry point. This script pulls everything together: your endpoints, your logic, and the server engine.

Summary: The Web Stack

The Renaissance approach to the web favors `Types over Strings`.

- `High Performance`: Powered by Netty's non-blocking I/O.
- `Type Safe`: If it compiles, your API shapes are guaranteed.
- `Interactive`: Swagger UI is generated for free.
- `Zero Boilerplate`: No XML, no complex config files.

Running from the Command Line

In the Renaissance ecosystem, we want the distance between "writing code" and "running a server" to be as short as possible. Because we are using `scala-cli`, you don't need to install a complex build tool or configure an IDE to see your API in action.

The Project Configuration

To ensure all the parts we've discussed (Tapir, Netty, Doobie, and sttp) are available, we create a `project.scala` file in the same directory as our source code. This acts as our manifest.

```
//> using scala "3.3.1"
//> using dep "com.softwaremill.sttp.tapir::tapir-netty-server:1.9.0"
//> using dep "com.softwaremill.sttp.tapir::tapir-json-zio:1.9.0"
//> using dep "com.softwaremill.sttp.tapir::tapir-swagger-ui-bundle:1.9.0"
//> using dep "com.softwaremill.sttp.client3::core:3.9.0"
//> using dep "org.tpolecat::doobie-core:1.0.0-RC4"
//> using dep "org.tpolecat::doobie-postgres:1.0.0-RC4"
```

Starting the Server

Once your code and configuration are in place, you can launch the server with one command. Scala-cli will automatically download the libraries, compile the code, and execute the \texttt{@main} method.

```
# Run the application
scala-cli run .
```

The Hot-Reload Workflow

For the best developer experience, use the \texttt{--watch} flag. This tells the compiler to stay active. Every time you save a change in your editor—perhaps to add a new API field or change a database query—the server will automatically re-compile and restart in less than a second.

```
# Run with automatic restarts on save
scala-cli run . --watch
```

Testing the Live API

Once the server reports it is "Started," you can interact with it immediately. You have three primary ways to verify it's working:

- **The Browser**: Visit `http://localhost:8080/docs` to see the interactive Swagger UI.

- **The Terminal**: Use `curl "http://localhost:8080/hello?name=Pragmatist"` to see a raw response.
- **The REPL**: Run `scala-cli repl .` and use `sttp` to call your own server programmatically.

Reviewer's Notes: This stack represents the "Holy Trinity" of modern Scala web development. It's safe, fast, and self-documenting.

Chapter 18

Extending Scala

Java Interoperability

One of Scala's greatest strengths is its zero-overhead interoperability with Java. Because Scala and Java both run on the JVM and share the same underlying type system at the bytecode level, a Java class is simply another type to Scala.

There is no need for wrappers, foreign function interfaces (FFIs), or translation layers. Scala code can call Java libraries directly and extend Java classes without any additional glue code.

This interoperability is strategically important. It gives Scala developers immediate access to more than twenty-five years of mature JVM libraries. If a Scala-specific library does not exist for a task, the Java equivalent can usually be used without modification.

Using Java Libraries from Scala

Calling Java code from Scala is completely transparent. You import the classes and use them just as you would in Java.

```
import java.time.LocalDateTime
import java.time.format.DateTimeFormatter
val now = LocalDateTime.now()
val formatted = now.format(DateTimeFormatter.ISO_DATE)
```

Because the runtime is shared, Java libraries often feel like a natural extension of the Scala language.

Extending Java Classes in Scala

Scala classes can directly extend Java classes or implement Java interfaces. This is particularly useful when working with frameworks that expect subclassing.

```
class MyServlet extends javax.servlet.http.HttpServlet:
override def doGet(req: HttpServletRequest, resp: HttpServletResponse): Unit
resp.getWriter.write("Hello from Scala via Java Servlet!")
```

Scala compiles to standard JVM bytecode, which allows Java frameworks to treat Scala classes exactly like Java ones.

Collection Interoperability

The primary friction between Java and Scala arises from their collection libraries. A java.util.List is not the same as a Scala List.

Scala provides built-in converters to bridge the two systems.

```
import scala.jdk.CollectionConverters.*
val javaList = new java.util.ArrayList[String]()
javaList.add("Java Item")
val scalaList = javaList.asScala
```

These converters allow Java collections to be viewed and manipulated using Scala's collection APIs.

Asynchronous Interoperability

Modern Scala programs frequently rely on asynchronous programming. Java and Scala both support asynchronous execution, but their primary abstractions differ.

Java uses:

```
java.util.concurrent.CompletableFuture
```

Scala uses:

```
scala.concurrent.Future
```

Scala provides conversion utilities that allow these types to interoperate.

Code Example

```
import scala.jdk.FutureConverters.*
val scalaFuture =
javaLibrary.fetchDataAsync().asScala
```

This allows Java asynchronous APIs to integrate seamlessly into Scala applications.

In systems built with functional effect libraries such as Cats Effect or ZIO, Java futures can also be lifted into effect types.

Code Example

```
IO.fromCompletableFuture(IO(javaLibrary.fetchDataAsync()))
```

Designing Java Libraries for Scala

When writing Java code intended to be used from Scala, a few design principles help maintain a clean integration.

Prefer Functional Interfaces

Java functional interfaces such as java.util.function.Function work very well with Scala due to Single Abstract Method (SAM) conversion. Scala developers can treat these interfaces as native lambda expressions.

Avoid Returning Null

Null values are one of the most common causes of runtime failures in mixed Java-Scala code.

Instead of returning `null`, prefer:

`java.util.Optional`

Using annotations such as `@Nullable` and `@NotNull` can also help static analysis tools detect potential problems before runtime.

Design Collection APIs Carefully

Scala developers often prefer immutable data structures.

A pragmatic approach when designing Java APIs is:

- Accept mutable Java collections such as `java.util.Collection`
- Return immutable views such as `List.of()` or `Collections.unmodifiableList()`

Scala users can then convert these results into Scala collections when needed.

Integration with the Scala Ecosystem

Well-designed Java libraries can integrate smoothly with the broader Scala ecosystem.

Networking with sttp

If a Java library implements low-level protocol or networking logic, expose extension points using functional interfaces.

Scala's SAM conversion allows these interfaces to be wrapped directly with Scala lambdas, making the Java code feel native when used with HTTP clients such as sttp.

Database Access with Doobie

The functional database library Doobie works by lifting JDBC operations into a functional context.

For interoperability, avoid hiding JDBC connections inside a Java library. Instead, design methods that accept a `java.sql.Connection`.

This allows a Scala application to pass a connection managed by a Doobie Transactor, preserving transactional guarantees across both languages.

When Writing Java Makes Sense

Although Scala can express everything Java can, there are situations where writing a small portion of a project in Java may still be useful.

Team Expertise

Some teams have deep Java experience, making certain components easier to maintain in Java.

Micro-Optimizations

In rare cases, very low-level array or memory operations may be easier for the JVM's just-in-time compiler to optimize in Java.

Framework Constraints

Older frameworks that rely heavily on reflection or strict Java conventions may be easier to satisfy using a small Java adapter class.

Best Practices Checklist

When designing Java code intended for Scala users:

- Use `Optional` instead of returning null
- Prefer functional interfaces for callbacks
- Ensure implementations are thread-safe
- Accept Java collections but return immutable views
- Keep the Java layer thin and focused on core logic
- Allow Scala code to handle effects such as retries, error handling, and concurrency

Part III

Scala Distilled

Chapter 19

From the Ground Up

The Renaissance Philosophy

Before we dive back into the mechanics of the language, we must understand the core pillars that make Scala 3 unique. It is not just "Java with better syntax" or "Haskell on the JVM." It is a multi-paradigm language designed to grow with you—from a one-line script to a global-scale distributed system.

The Three Pillars of Scala

Scala's power comes from the fusion of three distinct programming philosophies:

1. **Functional Programming (FP)**: Treating computation as the evaluation of mathematical functions and avoiding changing-state and mutable data.
2. **Object-Oriented Programming (OOP)**: Organizing code into "blueprints" (classes and traits) that encapsulate data and behavior.
3. **Strong Static Typing**: A rigorous compiler that acts as a first-line tester, catching errors before the code ever runs.

Conciseness: The End of Boilerplate

The most immediate change for a developer moving to Scala 3 is the **Significant Indentation** (Python-style syntax). By removing braces and semi-

colons, the logic of your program becomes the visual focus.

```
// A complete, readable class and logic in Scala 3
case class User(name: String, role: String)
  def greet(user: User): String =
    if user.role == "Admin" then
      s"Welcome back, Commander ${user.name}"
    else
      s"Hello, ${user.name}"
```

Immutability by Default

In the Renaissance approach, we favor `val` over `var`. By making data immutable, we eliminate a whole category of "spooky action at a distance" bugs. When you want to change data, you don't mutate it; you transform it into a new version.

Expressions, Not Statements

In many languages, an `if` block or a `try` block is a "statement" that does something. In Scala, almost everything is an **expression** that returns a value.

```
// The result of the 'if' is assigned directly to the variable
val status = if temperature > 30 then "Hot" else "Comfortable"
```

The Scalable Language

The name "Scala" comes from *SCAlable LAnguage*. This refers to the language's ability to create small, "internal" Domain Specific Languages (DSLs). This is why libraries like `sttp` and `Doobie` feel like they are part of the language itself rather than external additions.

Unified Type System

Unlike Java, which has "primitive" types (like `int`) and "objects" (like `Integer`), Scala treats everything as an object. However, the compiler is smart enough

to optimize these down to primitives under the hood to ensure there is no performance penalty.

Scala Crystallized—A Language at a Glance

This chapter serves as a formal map of the Scala language. It covers the structure of source files, the type system hierarchy, the nature of expressions, and the mechanics of error handling.

1. Source and Program Layout

Scala source files typically end in `.scala`. Unlike Java, the file name does not have to match the class name, and multiple classes, traits, and objects can reside in a single file.

- **Packages**: Defined at the top of the file. Scala supports "scoped" packages: `package com.project.core`.
- **Imports**: Can occur anywhere (top of file, inside a class, or inside a method).
  - `import scala.collection.mutable._` (Wildcard)
  - `import java.util.{List, Map}` (Selective)
  - `import java.sql.{Date => SqlDate}` (Renaming)
- **The Entry Point**: A program starts in an `object` (a singleton) with a `main` method or by extending `App`.

The Anatomy of a Scala Source File

1. Header: Packages and Imports

   Unlike Java, Scala allows "chained" package declarations and imports that can be scoped to specific blocks.

   - **Package Declaration**: Defines the logical namespace.
   - **Imports**: These are not restricted to the top of the file, but standard practice places them here. Scala supports renaming and hiding members during import to avoid name collisions (common when mixing Java and Scala libraries).

```
package com.adventure.service
import java.util.{UUID => JUUID} // Renaming for clarity
import scala.concurrent.ExecutionContext
import sttp.client3._ // Wildcard import
```

2. The Main Body: Traits, Classes, and Objects

   A single file often contains three distinct but related components. This is a departure from Java's "one-file-one-public-class" rule.

   (a) The Interface (The Trait)
   We define the "what" before the "how." In a library context, this is often a trait that defines the contract.

   (b) The Implementation (The Class)
   The concrete logic. Scala classes typically use **Constructor Parameters** instead of field declarations with setters.

   (c) The Companion Object
   This is where "static" logic lives. It shares the same name as the class. It is the idiomatic home for:

   - Factory methods (`apply`).
   - Data mappers (e.g., mapping a Doobie `Row` to a `User` class).
   - Constants specific to that type.

A Complete "Pragmatic" Example

Here is how a single file—let's call it `UserService.scala`—is structured in a project using **sttp** and **Doobie**.

```
package com.adventure.api
// 1. Imports
import doobie._
import doobie.implicits._
import sttp.model.Uri
// 2. The Domain Model (Data)
// Using a Case Class for immutability and built-in methods
case class User(id: Int, name: String, email: String)
```

```
// 3. The Service Trait (Contract)
// Defines the algebra of what our service can do
trait UserService {
  def findUser(id: Int): ConnectionIO[Option[User]]
}
// 4. The Implementation
// Logic is injected via the constructor
class UserServiceImpl(remoteApi: Uri) extends UserService {
  override def findUser(id: Int): ConnectionIO[Option[User]] = {
    // Doobie SQL program logic goes here
    sql"select id, name, email from users where id = $id".query[User].opt
  }
}
// 5. The Companion Object (Statics & Factories)
object UserService {
  // A factory method to hide the "new" keyword
  def apply(apiUri: Uri): UserService = new UserServiceImpl(apiUri)

  // A constant used by the service
  val MaxRetryCount = 3
}
```

3. The "Crystallized" Formatting Rules

   To maintain the "Pragmatic" style, follow these internal layout conventions:

Element	Rule	
Methods	Use def even for no-arg methods if they perform I/O.	
Spacing	One blank line between methods; two between classes/objects.	
Return Types	Always explicitly type public methods.	Prevent
Braces	Optional in Scala 3, but { ... } is standard for Scala 2.	

Summary of File Flow

1. **Identity**: Who am I? (`package`)
2. **Dependencies**: What do I need? (`import`)
3. **Shape**: What data do I hold? (`case class`)

4. **Behavior**: What can I do? (`trait & class`)
5. **Utilities**: What tools do I provide? (`object`)

2. The Type System Hierarchy

Scala has a unified type system. All types descend from a single root, `Any`.

- **AnyVal**: Primitive-like types (`Int`, `Double`, `Boolean`, `Unit`). These are often optimized into JVM primitives.
- **AnyRef**: All reference types (equivalent to `java.lang.Object`).
- **Nothing**: The "bottom type." It is a subtype of everything. A method that throws an exception or loops infinitely returns `Nothing`.
- **Null**: A subtype of all `AnyRef` types. It exists for Java interoperability, but its use is discouraged in idiomatic Scala in favor of `Option`.

3. Definitions and Variables

Scala distinguishes between immutable and mutable storage at the keyword level.

- `val` (**Value**): Immutable reference. Once assigned, it cannot be changed. Equivalent to `final` in Java or a constant in Ruby.
- `var` (**Variable**): Mutable reference. Use is generally discouraged in functional Scala.
- `def` (**Method**): Evaluated on every call.
- `lazy val`: Evaluated only once, the first time it is accessed.

4. Expressions and Control Flow

In Scala, **everything is an expression**. Unlike Ruby or Java, there are no "statements" that don't return a value.

- **If/Else**: Returns a value. `val result = if (x > 0) "pos" else "neg"`

- **For-Comprehensions**: More than a loop; it is syntactic sugar for `flatMap`, `map`, and `withFilter`.

```
result = for {
  user <- findUser(id) // Option[User]
  order <- getOrder(user) // Option[Order]
} yield order
```

- **Pattern Matching**: The "Swiss Army Knife" of Scala.

```
target match {
  case 1 => "One"
  case s: String => s"It's a string of length ${s.length}"
  case User(name, _) => s"Found user $name"
  case _ => "Default case"
}
```

5. Functional Foundations

- **Traits**: Similar to Java Interfaces but can contain concrete fields and methods. They are the primary unit of modularity (used extensively in `sttp` backends).
- **Case Classes**: Purpose-built for immutable data. They automatically provide `equals`, `hashCode`, `toString`, and `copy` methods.
- **Objects (Singletons)**: Used for static-like methods and the "Companion Object" pattern.

6. The "Magic" of Implicits and Contextual Abstraction

Scala 2 uses `implicit`, while Scala 3 uses `given/using`. This is how **Doobie** maps database rows to Scala types without manual boilerplate.

- **Implicit Parameters**: Allows the compiler to "plug in" a value (like a thread pool or a database transaction) automatically.
- **Extension Methods**: Allows you to add methods to existing classes (even Java classes) without inheritance.

7. Exception Handling and Effects

While Scala supports `try/catch/finally`, the pragmatic community (especially those using `Doobie` and `sttp`) treats exceptions as **values**.

- **Standard Try/Catch**:

```
try {
  unsafeCall()
} catch {
  case ex: IOException => println("Failed")
}
```

- **The Functional Alternative**:
  - `Option[T]`: For values that might be missing.
  - `Either[L, R]`: For operations that can fail with an error (`Left`) or succeed (`Right`).
  - `Try[T]`: A wrapper around a block that might throw an exception.

In the Scala world—especially when using libraries like sttp for networking or Doobie for database access—we treat errors as data, not as "explosions" (exceptions). Instead of jumping out of the program flow with a throw, we wrap the result in a container that describes the outcome.

Here is the breakdown of the "Big Three" error-handling types in Scala.

1. Option[T] — The "Maybe" Type

Use Case: When a value might simply be missing (replaces `null`).

The Logic: It has two subclasses: `Some(value)` and `None`.

- **When to use it**: Finding a record by ID, optional configuration fields, or a search result.
- **The Downside**: It doesn't tell you *why* the value is missing.

```
def findUser(id: Int): Option[String] = {
val db = Map(1 -> "Alice", 2 -> "Bob")
```

```
db.get(id) // returns Some("Alice") or None
}
// Usage
findUser(1) match {
case Some(name) => println(s"Found: $name")
case None => println("User not found")
}
// Functional approach (Transforming the value if it exists)
val shoutName = findUser(1).map(_.toUpperCase).getOrElse("ANONYMOUS")
```

2. `Try[T]` — The "Safety Net"

Use Case: When you are calling "dirty" code (usually Java libraries) that might throw a runtime exception.

The Logic: It has two subclasses: `Success(value)` and `Failure(exception)`.

- **When to use it:** Parsing a string to an integer, reading a file, or any operation where a traditional JVM exception is likely.
- **The Benefit**: It "catches" the exception and turns it into a value you can pass around.

```
import scala.util.{Try, Success, Failure}
def parseConfig(json: String): Try[Int] = Try {
  // This might throw a NumberFormatException
  json.toInt
}
// Usage
parseConfig("123") match {
  case Success(port) => println(s"Connecting to port $port")
  case Failure(ex) => println(s"Failed to parse config: ${ex.getMessage}")
}
```

3. Either[L, R] — The "Business Logic" Type

Use Case: When an operation can fail, and you need to provide a specific reason for that failure (the "Left" value).

The Logic: By convention, **Right** is "Right" (Success), and **Left** is for errors.

- **When to use it**: Domain logic. For example, **sttp** returns **Either[ResponseException, T]** because a 404 error is a different *type* of data than a 500 error.
- **The Benefit**: You can define a custom error hierarchy (ADT) for the Left side.

```
sealed trait RegistrationError
case object EmailTaken extends RegistrationError
case class InvalidPassword(reason: String) extends RegistrationError
def registerUser(email: String): Either[RegistrationError, String] = {
  if (email == "taken@job.com") Left(EmailTaken)
  else Right("User Account Created")
}
// Usage in a for-comprehension (Chaining multiple Eithers)
val result = for {
  _ <- registerUser("new@job.com")
  status <- Right("Success") // Pretend this is another check
} yield status
println(result) // Right("Success")
```

Summary Comparison

Type	Success	Failure	Primary Purpose
Option[T]	Some(v)	None	Handling optionality (Presence/Absence).
Try[T]	Success(v)	Failure(Throwable)	Wrapping legacy Java/exception-prone cod
Either[L, R]	Right(v)	Left(e)	Complex business logic with specific error typ

Chaining with for-comprehensions

One of the most powerful features of these types is that they all support flatMap. This allows you to chain operations together. If any step fails (returns None, Failure, or Left), the subsequent steps are skipped:

```
val finalResult = for {
  id <- parseConfig("101").toOption // Convert Try to Option
  name <- findUser(id) // Get Option[String]
  message <- registerUser(name).toOption // Convert Either to Option
} yield s"Processed: $message"
```

8. Feature summary

Feature	Scala
Top-level root	Any
Method Return	Last expression
Constants	val name = value
Missing values	None (Option)
Mixins	Traits
Anonymous Fn	x

Summary

When writing Scala, your source layout should prioritize **immutability** (`val` and `case class`) and **expression-oriented logic**. By treating error handling as data (`Either`) rather than control flow (`throw`), you align with the design of modern libraries like `sttp` and `Doobie`.

Chapter 20

Core Constructs

The Pragmatic "Other" Constructs

Beyond the classic battle between Objects and Functions lies a set of constructs that make Scala truly "The Scalable Language." These features allow us to define domain-specific rules, optimize performance, and handle data with surgical precision.

Opaque Type Aliases

Sometimes you want the performance of a simple type (like a `String` or `Int`) but the safety of a full class. Opaque types allow you to wrap a value so the compiler treats it as a unique type, but at runtime, the wrapper vanishes entirely.

```
object Domain:
  opaque type AccountId = String
object AccountId:
  def apply(id: String): AccountId = id
extension (id: AccountId)
  def value: String = id
// Outside, you cannot use AccountId as a String accidentally!
```

Enums (ADTs)

Scala 3 enums are far more than just lists of constants. They are Algebraic Data Types (ADTs). They allow you to define a fixed set of states, where

each state can carry its own specific data.

```
enum ConnectionStatus:
  case Connected(ip: String)
  case Disconnected(reason: String)
  case Connecting
```

Extension Methods

In many languages, if you want to add a method to a class you didn't write (like `String`), you're out of luck. In Scala, you can "extend" existing types with new functionality without modifying their source code.

```
extension (s: String)
  def shout: String = s.toUpperCase + "!!!"
  // Usage: "hello".shout returns "HELLO!!!"
```

Trait as Extensions

In Scala, traits are used to define modular components that can be "mixed in" to a class to extend its functionality. Unlike traditional single inheritance, a class can mix in multiple traits, allowing you to compose behavior from several distinct sources without the complexity of multiple inheritance.

When a trait is added to a class using the with keyword (or extends for the first trait), the class gains all the methods and fields defined in that trait. This is a powerful way to implement cross-cutting concerns like logging, validation, or authentication across different parts of an application.

Mixin Composition Example

In this example, the Robot class inherits its core identity from Machine but gains specific capabilities through mixins.

```
trait Flying {
  def fly(): Unit = println("Taking off into the clouds!")
}
trait Speaking {
```

```
  def sayHello(name: String): Unit = println(s"Hello, $name. I am operational
}
abstract class Machine(val model: String)
// Composing the class by mixing in multiple traits
class Drone(model: String) extends Machine(model) with Flying with Speaking
val myDrone = new Drone("RX-9")
myDrone.sayHello("User") // From Speaking
myDrone.fly() // From Flying
```

Key Concepts

- **Linearization**: Scala resolves method overlaps by looking at the traits from right to left.
- **Reusability**: Traits allow you to define a behavior once and apply it to any class, regardless of its position in the inheritance hierarchy.

Contextual Abstractions (Given/Using)

This is Scala's answer to Dependency Injection. It allows the compiler to "fill in the blanks" for you. If a function needs a specific context (like a Database connection or a JSON encoder), the compiler looks for a "given" instance in the current scope and passes it automatically.

```
case class Config(port: Int)
def startServer(using conf: Config) =
  println(s"Starting on ${conf.port}")
given standardConfig: Config = Config(8080)
startServer // The compiler sees the 'given' and inserts it here
```

Pattern Matching on Types (The Match Type)

Scala can perform logic not just on values, but on types themselves. This allows for extremely advanced "Generic" programming where the return type of a function changes based on the input type.

```
type Elem[X] = X match
  case String => Char
```

```
  case Array[t] => t
  case Iterable[t] => t
  // Elem[String] is Char, Elem[List[Int]] is Int
```

The Export Clause

Instead of writing "wrapper" methods to delegate calls to an internal object, you can use `export`. This "promotes" the methods of an internal object to the public interface of the current class.

```
class Printer:
  def print(msg: String) = println(msg)
class Office:
  val p = Printer()
  export p.print // Office now has a print() method automatically
```

Chapter 21

Functional Things

The Functional Core

Functional Programming (FP) in Scala is a way of writing programs by combining **pure functions** and **immutable data**. By treating logic as a series of transformations rather than a series of state changes, we create "Honest" code that does exactly what its signature says it will do.

Purity: No Spooky Action at a Distance

A function is "pure" if it always returns the same output for the same input and has no side effects (like modifying a global variable or writing to a database). This makes your code incredibly easy to test because you don't need to "set up the world" before calling a function.

```
// Pure: Predictable and honest
def add(a: Int, b: Int): Int = a + b
// Impure: Unpredictable (depends on external state)
var total = 0
def addToTotal(a: Int): Unit = total += a
```

Immutability: The Power of Persistent Data

In the functional world, we don't "change" an object; we create a new version of it. This is why `case classes` provide a `copy` method. This prevents bugs where one part of your program accidentally breaks another part by changing shared data.

```
case class Point(x: Int, y: Int)
val p1 = Point(1, 2)
val p2 = p1.copy(x = 10) // p1 remains (1, 2), p2 is (10, 2)
```

Higher-Order Functions: Functions as Data

In Scala, functions are "first-class citizens." You can pass a function into another function as an argument, or return one as a result. This allows us to write highly generic logic for processing data.

```
val numbers = List(1, 2, 3, 4, 5)
// map, filter, and fold are the "Big Three" of FP
val doubled = numbers.map(n => n * 2)
val evens = numbers.filter(n => n % 2 == 0)
val sum = numbers.foldLeft(0)((acc, n) => acc + n)
```

Pattern Matching: Deconstructing Reality

Pattern matching is the functional equivalent of a `switch` statement on steroids. It allows you to "un-apply" data structures to look inside them. It is the primary way we handle different shapes of data in functional Scala.

```
sealed trait Shape
  case class Circle(radius: Double) extends Shape
  case class Rect(w: Double, h: Double) extends Shape
    def area(s: Shape): Double = s match
  case Circle(r) => Math.PI * r * r
  case Rect(w, h) => w * h
```

Managing Effects: The Option and Either types

Functional programmers don't like throwing exceptions because they break the flow of the program. Instead, we represent "Errors" or "Absence" as data types. This forces the caller to acknowledge the possibility of failure at compile time.

```
// Option: Represents a value that might not be there
def findUser(id: Int): Option[User] = ???
```

```
// Either: Represents a result or an error
def divide(a: Int, b: Int): Either[String, Int] =
  if b == 0 then Left("Cannot divide by zero")
  else Right(a / b)
```

Referential Transparency

The "Gold Standard" of FP. If you can replace a function call with its resulting value without changing the behavior of your program, your code is **Referentially Transparent**. This makes refactoring a breeze—you can move code around with total confidence.

Chapter 22

Classes & Objects

Chapter 22: Classes, Objects, and Traits

While Functional Programming handles the logic of transformations, Object-Oriented Programming (OOP) handles the **architecture** of our data. In Scala, every value is an object, and every operation is a method call. This chapter explores how we define these entities and how they collaborate.

Classes: The Basic Blueprints

A class in Scala is more than just a container for data. It defines a type and the behavior associated with it. Unlike many other languages, Scala 3 allows for a very concise syntax where the constructor is defined right in the class header.

```
class Project(val name: String, var budget: Int):
  def increaseBudget(amount: Int): Unit =
    budget += amount
    println(s"New budget for $name is $budget")
    val p = Project("Renaissance", 1000)
```

Case Classes: Data Carriers

For most of your domain modeling, you will use `case classes`. These are specialized classes that automatically provide structural equality (comparing by data, not memory address), a readable `toString`, and the `copy` method we saw in the functional chapter.

```
case class Task(description: String, priority: Int)
val t1 = Task("Write Chapter 21", 1)
val t2 = Task("Write Chapter 21", 1)
// t1 == t2 is true!
```

Traits: The Ultimate Composables

Traits are the "Renaissance" version of Interfaces. They allow you to define a set of methods that a class must implement, but unlike Java interfaces, they can also contain fields and implemented logic. You "mix" them into classes to build complex behavior from simple parts.

```
trait Searchable:
  def slug: String
trait Auditable:
  def lastModified: Long = System.currentTimeMillis()
class Document(val title: String) extends Searchable with Auditable:
  def slug = title.toLowerCase.replace(" ", "-")
```

Objects: Singletons and Companions

In Scala, we don't have `static` members. Instead, we have the `object` keyword, which defines a **Singleton**—a class that has exactly one instance. When an object shares the same name as a class, it is called a **Companion Object**.

```
class DatabaseConnection(val url: String)
object DatabaseConnection:
  // A factory method in the companion object
  def local: DatabaseConnection =
    new DatabaseConnection("jdbc:postgresql://localhost:5432/db")
```

Opaque Types vs. Classes

As we saw in Chapter 19, sometimes a class is "too heavy" for a simple ID or wrapper. We use `opaque type` when we want the type safety of a class without the memory overhead of creating an actual object instance.

Inheritance and Open Classes

Scala 3 encourages composition over inheritance. By default, classes are "non-open." If you want someone to be able to inherit from your class, you must explicitly mark it as `open`. This prevents the "Fragile Base Class" problem where changes in a parent class unexpectedly break children.

```
open class BaseRenderer:
  def render(): Unit = println("Default render")
class WebRenderer extends BaseRenderer:
  override def render(): Unit = println("HTML render")
```

Reviewer's Notes: By combining Case Classes for data and Traits for behavior, Scala developers create systems that are incredibly modular and easy to refactor.

Chapter 23

Data Safety

From Ruby Taint to Scala Types

Ruby developers will remember `$SAFE` levels and "Tainted" objects—a system where the interpreter tracked whether a string came from a risky source (like a web form) and blocked it from reaching dangerous methods (like `eval`).

In Scala, we achieve "Safe Levels" through **Type-Level Domain Modeling**. Instead of the runtime checking a "taint flag" on a string, the compiler ensures that untrusted data is wrapped in a type that the rest of the system refuses to touch.

The Ruby Way: Runtime Tracking

In Ruby, a string is just a string, but it might have a hidden "taint" bit flipped.

The Scala Way: Type Separation

In Scala, we don't use flags. We use **Opaque Types** (from Chapter 19). We define a type for "Raw Input" and a type for "Sanitized Data." A function that executes SQL will *only* accept `SanitizedData`.

```
object Security:
  opaque type RawInput = String
  opaque type SanitizedData = String
```

```
  def markAsRaw(s: String): RawInput = s
  def sanitize(input: RawInput): SanitizedData =
    input.replaceAll("[';]", "") // Basic example
  // Your Database logic
  def executeQuery(query: SanitizedData): Unit = ???
// This fails to compile! You can't pass RawInput where Sanitized is needed.
val userPath = Security.markAsRaw("'; DROP TABLE Users;--")
executeQuery(userPath)
```

Implicit Safety: The "Taint" of Options

The closest spiritual successor to Ruby's "Safe Levels" in Scala is how we handle `null`. In Ruby, calling a method on `nil` causes a crash. In Scala's **Strict Nulls** mode (Chapter 23), the compiler "taints" any value that could be null by forcing it into a `String | Null` union type.

Taint Checking via Contextual Abstractions

We can also replicate "Safe Levels" using `given` and `using`. We can define a "Security Capability" that must be present in the scope for certain actions to be allowed.

```
case class SecurityLevel[L <: Int]()
// This function only runs if the "Level 2" capability is in the room
def deleteSystemFile()(using SecurityLevel[2]): Unit =
  println("File deleted")
// If this 'given' isn't in scope, the code above won't even compile
given currentLevel: SecurityLevel[1] = SecurityLevel[1]()
```

Summary: Prevention vs. Detection

- **Ruby Taint**: A "Safety Net" that catches you when you make a mistake at runtime.
- **Scala Types**: A "Guardrail" that prevents you from even steering toward the cliff at compile-time.

Reviewer's Notes: Moving security from runtime to compile-time is the ultimate "Pragmatic" move. It turns a potential 3:00 AM security breach into a 3:00 PM compiler error.

Chapter 24

Reflection & Metaprogramming

Reflection, Marshalling, and Metaprogramming

In the Renaissance era, we avoid "Runtime Reflection" because it is a black box that hides errors until the program crashes. Instead, we use **Derivation** and **Macros**. This allows the compiler to look at your classes and write the boring "glue code" for you.

Automatic Marshalling (Type-class Derivation)

Marshalling is the process of turning a memory object into a format like JSON or XML. In the past, this required slow reflection. In Scala 3, we use the `derives` keyword. The compiler "reflects" on the structure of your class at compile-time and generates the Marshaller automatically.

```
import zio.json.*
// The 'derives' keyword tells Scala to write the JSON logic for you
case class User(name: String, email: String) derives JsonCodec
val user = User("Alice", "alice@startup.io")
val json = user.toJson // "{"name":"Alice","email":"alice@startup.io"}"
```

Mirrors: The New Reflection

Scala 3 introduces `Mirror`. A Mirror is a compile-time representation of a class's shape. It tells you the names of the fields and their types without ever needing to "instantiate" the class. This is how libraries like `Doobie` map database rows to your Case Classes.

```
import scala.deriving.Mirror
// A Mirror lets you inspect Task without using Java Reflection
val m = summon[Mirror.Of[User]]
// m.MirroredElemLabels contains ("name", "email")
```

Inlining: Reflection without the Cost

The `inline` keyword is a request to the compiler to "copy-paste" the code at the call site. When combined with `if` statements, the compiler can perform "Constant Folding"—evaluating logic during compilation and removing dead code.

```
inline def log(msg: String): Unit =
  inline if Thread.currentThread().getName == "main" then
    println(s"[MAIN] $msg")
// If called outside 'main', the compiler removes the entire call!
```

Macros: Inspecting Code as Data

While Ruby uses `eval` to run strings as code, Scala 3 uses `Quotes` and `Splices` (as seen in Chapter 23). This is "Safe Reflection." You can inspect the AST (Abstract Syntax Tree) of a piece of code, validate it, and then transform it.

```
// A macro that prints the name of a variable at compile-time
inline def debug(inline expr: Any): Unit =
  ${ debugImpl('expr) }
// The implementation uses reflection to see the source code name
```

Pickling and Unpickling

In some contexts, you'll hear about "Pickling." This is a high-performance form of marshalling used for distributed systems (like Spark or Akka). It serializes objects into a binary format. Because Scala 3 has a unified type system, "unpickling" a binary stream back into a typed object is safer than ever.

Summary: The Death of the Runtime Error

The Renaissance philosophy of reflection is simple:

- **Marshalling**: Should be derived, not manually written.
- **Reflection**: Should happen at compile-time via Mirrors.
- **Metaprogramming**: Should be typed and quoted, never raw strings.

Part IV

References & Appendices

Chapter 25: The Standard Library Atlas

A language is only as useful as its toolkit. Scala's standard library is designed to be "batteries included" but "type-safe by default." This reference covers the essential modules you will touch every day.

1. Any, AnyVal, and AnyRef: The Hierarchy

At the top of the world sits `Any`. Everything in Scala inherits from it. It splits into two branches:

- **AnyVal**: Primitive types like `Int`, `Double`, `Boolean`, and `Unit`.
- **AnyRef**: All objects (the equivalent of `java.lang.Object`).

2. The Collection Module (`scala.collection`)

The "Crown Jewel" of the library. It is split into **immutable** (default) and **mutable** packages.

- **Seq / List**: Ordered sequences. Use `List` for functional recursion and `Vector` for random access performance.
- **Set**: Unordered collections of unique elements.
- **Map**: Key-value pairs.

```
val fruits = List("Apple", "Banana")
val prices = Map("Apple" -> 0.99, "Banana" -> 0.50)

// The power of the library is in the combinators:
val shoppingList = fruits.flatMap(f => prices.get(f).map(p => s"$f: $p"))
```

3. The Option, Either, and Try Modules

These are the "Safety" containers that replace `null` and `exceptions`.

- **Option**: `Some(value)` or `None`.
- **Either**: `Right(value)` (the "right" way) or `Left(error)`.

- **Try**: `Success(value)` or `Failure(exception)`. Use this when interacting with legacy Java code that throws.

4. Future and Concurrency (`scala.concurrent`)

Scala makes asynchronous programming "Pragmatic" using `Future`. It represents a value that will eventually be available.

```
import scala.concurrent.Future
import scala.concurrent.ExecutionContext.Implicits.global

val asyncTask = Future {
  Thread.sleep(1000)
  "Data Loaded"
}
```

5. StringOps: The String Extension

In Scala, a `String` is technically a `java.lang.String`, but Scala uses **Extension Methods** to give it functional powers like `.map`, `.filter`, and `.splitAt`.

6. Numeric and Math (`scala.math`)

Standard mathematical functions (sin, cos, max, min) and `BigDecimal` / `BigInt` for high-precision financial calculations.

7. The Predef Object

There is a secret object called `Predef` that is imported automatically into every Scala file. It provides:

- `println` (aliased from `Console.println`)
- `identity` (a function that returns its input)
- Type aliases like `String`, `Map`, and `Set`.

Chapter 25.1: The Digital Library (External References)

While this book covers the core patterns, the Scala ecosystem moves quickly. To be a truly effective Renaissance developer, you should keep these specific digital references at your fingertips.

The "Gold Standards" for Documentation

- **The Scala 3 Standard Library API (Scaladoc)**: This is the definitive source for every method on every class. Use the search bar to find combinators like `groupBy`, `zipWithIndex`, or `collect`. *Link: https://scala-lang.org/api/3.x/*
- **The Scala 3 Book (Online Version)**: An excellent, community-driven guide that goes deep into the specific syntax changes from Scala 2 to Scala 3. *Link: https://docs.scala-lang.org/scala3/book/introduction.html*

Library-Specific References (Our Recommended Stack)

Because we have standardized our application on **sttp**, **Doobie**, and **Tapir**, you will need their specific "Cheatsheets":

- **Tapir Documentation**: Essential for looking up different endpoint inputs (headers, cookies, multipart forms). *Link: https://tapir.softwaremill.com/*
- **sttp Client Reference**: Details on how to handle proxies, retries, and different backends (like Pekko or OkHttp). *Link: https://sttp.softwaremill.com/*
- **Doobie Microsite**: The go-to for learning how to map custom SQL types or handle complex transaction boundaries. *Link: https://tpolecat.github.io/doobie/*

Community and "Pragmatic" Help

- **Scala Times**: A weekly newsletter that tracks the most important library updates and blog posts in the ecosystem.

- **Scaladex**: The "Package Index" for Scala. If you are looking for a library to handle a specific task (e.g., "AWS SDK" or "CSV Parsing"), Scaladex will show you the most popular and up-to-date options for Scala 3. *Link: https://index.scala-lang.org/*

Visual Cheat Sheet: The Collection Hierarchy

Note: When in doubt, start with an immutable `List`. If you need to search by key, move to a `Map`. If you need uniqueness, use a `Set`. The standard library makes it trivial to convert between them using `.toList`, `.toMap`, or `.toSet`.